

Core Maintenance on Dynamic Multilayer Graphs

Yuhan Zhou¹, Chuanyu Liu¹, Qing Liu^{1,2}, Chengyang Luo¹

¹Zhejiang University, ²Zhejiang Key Laboratory of Big Data Intelligent Computing
{zhou_yh, liucy219, qingliucs, luocy1017}@zju.edu.cn

ABSTRACT

Multilayer core is a fundamental cohesive subgraph model for multilayer graph analysis. Existing multilayer core decomposition methods, which aim to compute all multilayer cores of a given multilayer graph, mainly focus on static graphs. However, real-world multilayer graphs evolve continuously. Recomputing all multilayer cores from scratch after each graph update is computationally expensive. Therefore, in this paper, we systematically explore the multilayer core maintenance problem on dynamic multilayer graphs. To address this problem, we propose efficient maintenance algorithms that first identify the affected scope for graph updates, and then compute the updated multilayer core through local convergence. To enhance efficiency, we reduce the affected scope through theoretical analysis and accelerate the local convergence procedure with a series of optimizations. Moreover, we extend our algorithms to batch updates with a hybrid edge grouping strategy, which enables all edges within the same group to be processed simultaneously. Extensive experiments on real-world multilayer graphs demonstrate that our proposed algorithms are up to 3 orders of magnitude faster than the state-of-the-art method.

PVLDB Reference Format:

Yuhan Zhou, Chuanyu Liu, Qing Liu, Chengyang Luo. Core Maintenance on Dynamic Multilayer Graphs. PVLDB, 14(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/ZJU-DAILY/MCM>.

1 INTRODUCTION

Real-world entities often exhibit multiple types of relationships [31, 50], which can be naturally modeled as multilayer graphs. In such graph structures, vertices are shared across different layers, while the edges in each layer represent a specific type of relationship [40, 53]. Multilayer graphs have been widely adopted in various application scenarios, such as social networks, collaboration networks, and financial networks [19, 34, 47]. Figure 1(a) shows an example of a multilayer graph.

As a fundamental task in graph analysis, cohesive subgraph mining has been extensively studied [12, 25, 28]. One of the most representative models is k -core, which requires each vertex in the subgraph to have at least k neighbors [5, 8, 46]. To extend the k -core model to multilayer graphs, Azimi et al. [2] proposed multilayer core, also called $\mathbf{k}$ -core, where $\mathbf{k} = [k_\ell]_{\ell \in L}$ requires that the

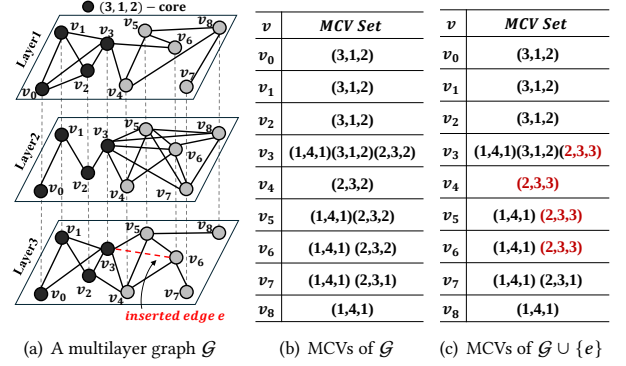


Figure 1: An example of multilayer core maintenance

subgraph induced on each layer $\ell \in L$ forms a k_ℓ -core (L is the layer set). In Figure 1(a), the subgraph induced by $\{v_0, v_1, v_2, v_3\}$ is a $\mathbf{k}$ -core with $\mathbf{k} = (3, 1, 2)$. The $\mathbf{k}$ -core has a wide range of applications [2, 15, 35], including social recommendation, collaboration analysis, and fraud detection.

To efficiently compute $\mathbf{k}$ -cores, previous studies [15, 35] investigated the multilayer core decomposition problem, which aims to enumerate all $\mathbf{k}$ -cores on a given multilayer graph $\mathcal{G}$ and return the maximal coreness vectors (MCVs) of each vertex [15]. A vector $\mathbf{k}$ is a coreness vector of a vertex v if v belongs to a $\mathbf{k}$ -core. For two vectors $\mathbf{k}$ and $\mathbf{k}'$, $\mathbf{k}$ is dominated by $\mathbf{k}'$ if $k_\ell \leq k'_\ell$ for all $\ell \in L$. A coreness vector is maximal if it is not dominated by any other coreness vector of the same vertex. Figure 1(b) illustrates the results of the multilayer core decomposition of $\mathcal{G}$. The MCV of v_4 is $\mathbf{k}^* = (2, 3, 2)$, indicating that it can be contained in any $\mathbf{k}$ -core with $\mathbf{k}$ dominated by $\mathbf{k}^*$. Based on the results of the multilayer core decomposition, we can efficiently retrieve the $\mathbf{k}$ -core by collecting the vertices with an MCV $\mathbf{k}^*$ dominating $\mathbf{k}$.

Despite the advances in existing studies, they are primarily designed for static multilayer graphs, whereas real-world networks are highly dynamic [16, 37, 48]. For example, user connections in social networks constantly change; researchers in collaboration networks frequently form new partnerships; and malicious actors in fraud networks continually adapt their interactions to evade detection. These dynamic changes highlight the need for efficient algorithms to update $\mathbf{k}$ -cores after graph changes. Therefore, for the first time, we study the multilayer core maintenance on dynamic multilayer graphs and theoretically analyze its update boundedness. Specifically, given a multilayer graph and an updated edge, the goal is to maintain the MCVs of vertices.

Multilayer core maintenance can benefit various dynamic graph analysis tasks by supporting efficient updates of cohesive structures in evolving multilayer networks. In co-purchasing networks [29], where layers denote item co-purchasing relations across different snapshots, it can quickly track evolving product groups as new transactions arrive, supporting dynamic product recommendation

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

and market trend analysis. In biological networks [42], where layers capture different types of genetic and protein interactions, it enables efficient updates of cohesive gene groups when new interactions are discovered, facilitating the analysis of functional modules and disease-related gene associations. In social networks [14], where layers capture interactions on different platforms, it supports timely tracking of evolving communities, helping reveal emerging influential groups and cross-platform interaction patterns.

A straightforward solution for multilayer core maintenance is to invoke existing multilayer core decomposition algorithms after each graph update. However, it is inefficient because it recomputes all k -cores from scratch, even though only a small fraction of vertices and edges are typically affected in real-world scenarios [55]. In the literature, various core maintenance algorithms [4, 7, 13, 23, 44] have been proposed for different types of graphs. However, none of them can be directly applied to our problem due to the difference in graph structures and core models.

Designing efficient algorithms for multilayer core maintenance poses substantial challenges. First, in multilayer graphs, a vertex may have numerous MCVs, and the number of such vectors can grow rapidly with the number of layers, significantly increasing the overhead of multilayer core maintenance. Second, multilayer cores exhibit complex cross-layer dependencies. A single-edge update may affect a large number of vertices through interlayer connections. Moreover, the MCVs of affected vertices may change simultaneously across multiple dimensions, further complicating the computation of updated MCVs. Third, unlike the scalar core-ness value in single-layer graphs, the MCV of a vertex is a vector. This multidimensional nature substantially increases the cost of fine-grained affected-vertex detection, while coarse-grained detection may lead to an excessively large candidate region of affected vertices.

To address these challenges, this paper proposes efficient multilayer core maintenance algorithms that first identify the affected scope for graph updates, and then compute the updated MCVs through local convergence. For the affected scope identification, we systematically analyze the respective impacts of edge insertion and deletion. For edge insertion, we prove that each updated MCV is dominated by an original MCV whose value is increased by one on the updated layer. Based on this bound, we initialize affected vectors from the endpoints of the inserted edge and iteratively propagate the incremental impact to all potentially influenced vertices. For edge deletion, we observe that the vector obtained by decreasing the deleted-layer value of the original MCV by one remains a valid core-ness vector in the updated graph. Based on this observation, we track original MCVs that may lose dominating MCVs from their neighbors, and iteratively propagate such losses using the valid core-ness vector as a pruning condition to identify a compact set of affected vertices. Consequently, the affected scope identification procedure returns the affected vertices and, for each of them, an upper-bound vector set that dominates all its updated MCVs.

After obtaining the affected vertices and their upper-bound vector sets, we compute the updated MCVs through a local convergence procedure. Starting from the upper-bound vectors, the procedure repeatedly validates candidate vectors and refines invalid ones until convergence. Specifically, an invalid vector is split greedily by decreasing the value of each dimension, and the resulting vectors

are further validated. The process terminates when all remaining vectors are valid MCVs. To improve efficiency, we optimize both validation and refinement. For validation, we reuse historical validation results and incrementally update the maintained dominance information. For refinement, we shrink the search space through repeated vector filtering, binary state skipping, and redundant vector pruning.

Furthermore, we extend our maintenance algorithm to support batch updates through a hybrid edge grouping strategy. The key observation is that multiple edge updates can be processed simultaneously if they preserve the update bound derived for a single-edge update. To this end, we first introduce disjoint edge groups, in which no two updated edges share an endpoint. Consequently, each vertex gains or loses at most one incident edge on the updated layer, ensuring that the single-edge update bound remains valid. To prevent performance degradation when many updated edges share common endpoints, we further introduce joint edge groups, which permit shared endpoints whose MCV values on the updated layer are strictly larger than those of their opposite endpoints. Based on these two grouping rules, we identify the affected scope and perform local convergence group by group to compute the updated MCVs efficiently.

The main contributions of this paper are as follows:

- We formulate the multilayer core maintenance problem for the first time, and theoretically analyze its boundedness.
- We devise efficient two-phase multilayer core maintenance algorithms and a set of optimization techniques, supported by thorough theoretical analysis.
- We design a hybrid edge grouping strategy for our maintenance algorithms to efficiently handle batch updates.
- We conduct extensive experiments on real-world multilayer graphs, demonstrating the efficiency of our proposed algorithms and the effectiveness of optimizations.

Roadmap. Section 2 reviews the related work. Section 3 formulates the problem. Section 4 presents the overview of our maintenance framework. Sections 5 and 6 detail the maintenance algorithms. Section 7 introduces the batch updates. Section 8 reports the experimental results. Finally, Section 9 concludes this paper.

2 RELATED WORK

Core maintenance. The core maintenance aims to update the core-ness of vertices when the graph is updated. The existing works can be categorized into traverse-based methods, k -order-based methods, and convergence-based methods. Traverse-based methods [30, 44, 45] exploit the property that the insertion or deletion of a single edge changes a vertex’s core-ness by at most one, and update the affected region through graph traversal. k -order-based methods [18, 57] further accelerate the process by utilizing the peeling order generated during core decomposition. Convergence-based methods [9, 17, 22] iteratively refine core-ness values based on whether neighboring vertices can support the current core-ness until convergence. For batch updates, several techniques [21, 27, 52] have been proposed to partition updated edges into independent batches, enabling more efficient processing. Core maintenance has also been studied on different types of graphs, including weighted graphs [56, 58], directed graphs [13, 32], bipartite graphs [4, 41, 55], uncertain graphs [7],

temporal graphs [3, 54], and hypergraphs [11, 23, 39]. Despite the advances of existing works, they are designed for specific graphs and core models. Therefore, they cannot be adapted to our multilayer core maintenance problem.

Cohesive subgraphs in multilayer graphs. Various multilayer cohesive subgraph models have been proposed, including core-based models [2, 15, 20, 33, 35, 36, 40, 47, 53, 60], truss-based models [6, 24, 38], and clique-based models [26, 51]. Among them, core-based models are one of the most representative ones. Specifically, the k -core with $\mathbf{k} = [k_\ell]_{\ell \in L}$ enforces that the subgraph induced in each layer $\ell \in L$ forms a k_ℓ -core [2, 15, 35], which is the focus of this paper. By relaxing these conditions or introducing additional constraints, several variants of the multilayer core have been proposed. For example, d -CC [33, 60] imposes the d -core constraint on a given subset of layers $L' \subseteq L$. FirmCore [20] ensures that each vertex has degree at least k in at least λ layers. FocusCore [53] further integrates the constraints of d -CC and FirmCore. In addition, gCore [36], (k, d) -core [47], and SynCore [40] incorporate cross-layer or projected-layer constraints. However, existing works only consider static multilayer graphs, whereas this paper focuses on dynamic multilayer graphs.

3 PROBLEM FORMULATION

A multilayer graph can be represented as $\mathcal{G} = (V, E, L)$, where V is the set of vertices, $E \subseteq V \times V \times L$ is the set of edges, and L is the set of layers. For each $\ell \in L$, the corresponding single-layer graph in layer ℓ is denoted by $\mathcal{G}_\ell = (V, E_\ell)$. Note that graphs on different layers share the same vertex set, while their edge sets differ from one another. Let $C \subseteq V$ be a subset of vertices. The induced multilayer subgraph of $\mathcal{G}$ on C is defined as $\mathcal{G}[C] = (C, E[C], L)$. Similarly, for each layer ℓ , the induced subgraph of $\mathcal{G}_\ell$ on C is denoted by $\mathcal{G}_\ell[C] = (C, E_\ell[C])$ with $E_\ell[C] = \{(u, v) \in E_\ell \mid u, v \in C\}$. For a vertex $v \in V$, we use $N_{\mathcal{G}}(v, \ell)$ and $\deg_{\mathcal{G}}(v, \ell)$ to denote v 's neighbors and degree in $\mathcal{G}_\ell$, respectively. Correspondingly, $N_{\mathcal{G}}(v) = \bigcup_{\ell \in L} N_{\mathcal{G}}(v, \ell)$. We use $e = (u, w, \ell)$ to denote a single updated edge on layer ℓ . The updated multilayer graph after inserting and deleting e is denoted by $\mathcal{G} \cup \{e\}$ and $\mathcal{G} \setminus \{e\}$, respectively. Next, we introduce the multilayer core and subsequently formulate the problem.

DEFINITION 1. (Multilayer $\mathbf{k}$ -core [2, 15, 35]). Given a multilayer graph $\mathcal{G} = (V, E, L)$ and an $|L|$ -dimensional integer vector $\mathbf{k} = [k_\ell]_{\ell \in L}$, the multilayer $\mathbf{k}$ -core of $\mathcal{G}$ is a subgraph $\mathcal{G}[C] \subseteq \mathcal{G}$ such that $\forall v \in C \subseteq V, \ell \in L, \deg_{\mathcal{G}[C]}(v, \ell) \geq k_\ell$.

DEFINITION 2. (Dominance Relations). Given two $|L|$ -dimensional integer vectors $\mathbf{k} = [k_\ell]_{\ell \in L}$ and $\mathbf{k}' = [k'_\ell]_{\ell \in L}$,

- (1) $\mathbf{k}$ dominates $\mathbf{k}'$ (denoted by $\mathbf{k} \succeq \mathbf{k}'$): $\forall \ell \in L, k_\ell \geq k'_\ell$;
- (2) $\mathbf{k}$ strongly dominates $\mathbf{k}'$ (denoted by $\mathbf{k} \succ \mathbf{k}'$): $\mathbf{k} \succeq \mathbf{k}'$ and $\exists \ell \in L, k_\ell > k'_\ell$;
- (3) if $\mathbf{k}$ does not dominate $\mathbf{k}'$, we denote it by $\mathbf{k} \not\succeq \mathbf{k}'$;
- (4) $\mathbf{k}$ and $\mathbf{k}'$ are incomparable: $\mathbf{k} \not\succeq \mathbf{k}'$ and $\mathbf{k}' \not\succeq \mathbf{k}$.

DEFINITION 3. (Maximal Coreness Vector (MCV) [15]). Given a multilayer graph $\mathcal{G} = (V, E, L)$, an $|L|$ -dimensional integer vector $\mathbf{k}^* = [k_\ell^*]_{\ell \in L}$, and a vertex $v \in V$,

- (1) $\mathbf{k}^*$ is a coreness vector of v if there exists a non-empty $\mathbf{k}^*$ -core in $\mathcal{G}$ containing v ;

- (2) $\mathbf{k}^*$ is a maximal coreness vector of v if there does not exist any coreness vector $\mathbf{k}'$ of v satisfying $\mathbf{k}' \succ \mathbf{k}^*$.

Note that a vertex v may possess multiple MCVs that are pairwise incomparable. We denote the set of all MCVs of v on graph $\mathcal{G}$ as $\Phi_{\mathcal{G}}(v)$. Given a multilayer graph $\mathcal{G}$, we can compute $\Phi_{\mathcal{G}}(v)$ for each vertex v using the multilayer core decomposition algorithms [15, 35]. When edges are inserted into or removed from $\mathcal{G}$, $\Phi_{\mathcal{G}}(v)$ may change, thereby calling for efficient maintenance strategies. Based on it, we formally define our problem below.

PROBLEM 1. (Multilayer Core Maintenance). Given a multilayer graph $\mathcal{G} = (V, E, L)$, an edge $e = (u, w, \ell)$ to be inserted/deleted, and $\Phi_{\mathcal{G}}(v)$ for each vertex v of $\mathcal{G}$, the problem of multilayer core maintenance is to compute the updated MCV set $\Phi_{\mathcal{G}'}(v)$ for each vertex v based on $\Phi_{\mathcal{G}}(v)$. Here, $\mathcal{G}' = (V, E', L)$ denotes the updated multilayer graph, where $E' = E \cup \{e\}$ for edge insertion and $E' = E \setminus \{e\}$ for edge deletion.

For example, Figure 1(b) illustrates the multilayer core decomposition result of $\mathcal{G}$ in Figure 1(a). The corresponding maintenance result after inserting edge e is shown in Figure 1(c), where the updated MCVs of vertices v_3, v_4, v_5 , and v_6 are highlighted in red.

Boundedness analysis. Following [32, 55], the multilayer core maintenance problem is bounded if there exists an algorithm whose update cost can be bounded by a polynomial function of the size of the c -hop neighbors of vertices whose MCV sets change; otherwise, it is unbounded. Next, we present a proof sketch showing that multilayer core maintenance is unbounded via a reduction from D-core maintenance, which is proved to be unbounded [32]. Given a directed graph G_D , a D-core, i.e., a (k, l) -core, is the subgraph in which every vertex has in-degree at least k and out-degree at least l . The key idea of the proof is to encode the in-degree and out-degree constraints of a D-core as degree constraints on two different layers of a multilayer graph $\mathcal{G}$. Specifically, for each directed edge in G_D , we create a copy of the same edge in both layers. The first layer captures incoming-neighbor constraints, where the degree of a vertex is defined as its in-degree, whereas the second layer captures outgoing-neighbor constraints, where the degree of a vertex is defined as its out-degree. Under this construction, each (k, l) -core in G_D corresponds exactly to a multilayer $\mathbf{k}$ -core with $\mathbf{k} = (k, l)$ in $\mathcal{G}$, and maintaining D-cores under edge updates can be reduced to maintaining multilayer cores. Therefore, if multilayer core maintenance were bounded, then D-core maintenance would also be bounded, contradicting the known unboundedness of D-core maintenance [32]. Hence, the multilayer core maintenance problem is unbounded.

Note. Due to space limitations, some detailed proofs of this paper are provided in the appendix of our technical report [59].

4 OVERVIEW OF ALGORITHM

In this section, we present the overview of our multilayer core maintenance algorithm. The maintenance algorithm consists of two phases: *affected scope identification* and *local convergence*. The first phase identifies candidate vertices whose MCVs may change, and the second phase computes the updated MCVs for these vertices.

Let $\bar{\Phi}_{\mathcal{G}}(v)$ be the set of incomparable upper-bound vectors of $\Phi_{\mathcal{G}}(v)$ such that for all $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$, there exists $\mathbf{k}' \in \bar{\Phi}_{\mathcal{G}}(v)$, $\mathbf{k}' \succeq \mathbf{k}$.

Algorithm 1: Multilayer Core Maintenance Overview

Input: A multilayer graph $\mathcal{G}$; input MCV sets $\Phi_{\mathcal{G}}(\cdot)$; updated edge $e = (u, w, \ell)$; operation $op \in \{\text{insert}, \text{delete}\}$

Output: Updated MCV sets $\Phi_{\mathcal{G}'}(\cdot)$

- 1 $\mathcal{G}' \leftarrow \mathcal{G} \cup \{e\}$ if $op = \text{insert}$; otherwise $\mathcal{G}' \leftarrow \mathcal{G} \setminus \{e\}$;
 - 2 $(\mathcal{A}(e), \bar{\Phi}_{\mathcal{G}'}(\cdot)) \leftarrow \text{IdentifyScope}(\mathcal{G}', \Phi_{\mathcal{G}}(\cdot), e, op)$;
 - 3 $\Phi_{\mathcal{G}'}(\cdot) \leftarrow \text{LocalConvergence}(\mathcal{G}', \bar{\Phi}_{\mathcal{G}'}(\cdot), \mathcal{A}(e))$;
 - 4 **return** $\Phi_{\mathcal{G}'}(\cdot)$;
-

We say $\mathbf{k}'$ is an upper-bound vector of $\mathbf{k}$. Algorithm 1 shows the overview of our maintenance algorithm. The operation op specifies whether e is inserted or deleted. Accordingly, the algorithm constructs the updated graph as $\mathcal{G}' = \mathcal{G} \cup \{e\}$ for insertion, or $\mathcal{G}' = \mathcal{G} \setminus \{e\}$ for deletion (line 1). It then calls `IdentifyScope` to obtain the affected vertex set $\mathcal{A}(e)$ and the upper-bound vector sets $\bar{\Phi}_{\mathcal{G}'}(\cdot)$ (line 2). Finally, the algorithm invokes `LocalConvergence` to refine the upper-bound vector sets into the updated MCV sets $\Phi_{\mathcal{G}'}(\cdot)$ and returns them (lines 3-4).

Next, we introduce the procedure for affected scope identification and local convergence.

5 AFFECTED SCOPE IDENTIFICATION

In this section, we analyze the affected scope for edge insertion and deletion, respectively, and then introduce the procedure to identify the affected vertices and their upper-bound vector sets.

5.1 Edge Insertion

In this subsection, we discuss the affected bound and affected scope under single-edge insertion.

Affected bound. For single-edge insertion, we observe that the MCV of each vertex can increase by at most one in the dimension corresponding to the inserted layer, while remaining unchanged in all other dimensions.

LEMMA 1 (EDGE INSERTION BOUND). *Let $\mathcal{G}' = \mathcal{G} \cup \{e\}$, where e is inserted on layer ℓ . For any vertex $v \in V$ and any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$, there exists an original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ such that $\mathbf{k} + \mathbf{e}_{\ell} \succeq \mathbf{k}'$, where $\mathbf{e}_{\ell}$ is the unit vector whose ℓ -th dimension is one and all other dimensions are zero.*

Based on Lemma 1, once the affected vectors have been identified, we can increase their ℓ -th value by one to construct upper-bound vectors. We next show how to identify the affected scope for single-edge insertion.

Affected scope. For an edge $e = (u, w, \ell)$ inserted on layer ℓ , we maintain an affected vertex set $\mathcal{A}(e)$ and an affected vector set $\mathcal{A}_{\Phi}(e, v)$ of vertex v . Given a vector $\mathbf{k} \in \mathbb{N}^{[L]}$, the subspace vector set of $\mathbf{k}$ is $\mathcal{D}(\mathbf{k}) = \{\mathbf{x} \in \mathbb{N}^{[L]} \mid \mathbf{k} \succeq \mathbf{x}\}$. It is noted that $\mathbf{k}$ is also included in its subspace vector set. We say a vector can be affected by an edge insertion if it is a maximal subspace vector of an original MCV that may become an updated MCV. The key question lies in how to identify these affected vectors. Next, we introduce two cases of affected vectors.

Affected vectors of inserted endpoints. At the endpoint of the inserted edge $e = (u, w, \ell)$, the subspace vector of the endpoint's MCVs is directly affected by the newly inserted edge. Specifically, for

$\mathbf{k} \in \Phi_{\mathcal{G}}(u)$ and $\mathbf{k}' \in \Phi_{\mathcal{G}}(w)$, let $\mathbf{r} = \mathbf{k} \wedge \mathbf{k}' = [\min\{k_{\ell}, k'_{\ell}\}]_{\ell \in L}$ be their component-wise minimum. If $\mathbf{r} + \mathbf{e}_{\ell}$ is not dominated by any original MCV of u , then $\mathbf{r}$ is a possible affected vector of u .

An MCV $\mathbf{k}$ of a vertex v is said to be supported by a neighbor $u \in N_{\mathcal{G}}(v)$ if there exists an MCV of u that dominates $\mathbf{k}$. Intuitively, the affected vector must be supported by the inserted neighbor w . For a pair of original MCVs $\mathbf{k} \in \Phi_{\mathcal{G}}(u)$ and $\mathbf{k}' \in \Phi_{\mathcal{G}}(w)$, $\mathbf{r} = \mathbf{k} \wedge \mathbf{k}'$ characterizes the maximal subspace vector of $\mathbf{k}$ that can be dominated by $\mathbf{k}'$, and thus may obtain a support from the inserted neighbor w . Since the insertion contributes at most one additional support on layer ℓ by Lemma 1, if $\mathbf{r} + \mathbf{e}_{\ell}$ is not dominated by any original MCV of u , then $\mathbf{r}$ is a possible affected vector; otherwise, $\mathbf{r} + \mathbf{e}_{\ell}$ cannot be an updated MCV. Therefore, we can initialize $\mathcal{A}_{\Phi}(e, u) = \{\mathbf{r} \mid \exists \mathbf{k} \in \Phi_{\mathcal{G}}(u), \exists \mathbf{k}' \in \Phi_{\mathcal{G}}(w), \mathbf{r} = \mathbf{k} \wedge \mathbf{k}', \nexists \mathbf{h} \in \Phi_{\mathcal{G}}(u) : \mathbf{h} \succeq \mathbf{r} + \mathbf{e}_{\ell}\}$ and $\mathcal{A}_{\Phi}(e, w) = \{\mathbf{r} \mid \exists \mathbf{k} \in \Phi_{\mathcal{G}}(w), \exists \mathbf{k}' \in \Phi_{\mathcal{G}}(u), \mathbf{r} = \mathbf{k} \wedge \mathbf{k}', \nexists \mathbf{h} \in \Phi_{\mathcal{G}}(w) : \mathbf{h} \succeq \mathbf{r} + \mathbf{e}_{\ell}\}$, respectively.

Affected vectors of other vertices. Based on $\mathcal{A}_{\Phi}(e, u)$ and $\mathcal{A}_{\Phi}(e, w)$, other vertices may be affected when this change is propagated through neighboring vertices. Specifically, after being increased by one on layer ℓ , an affected vector may newly dominate a promoted subspace vector of a neighboring MCV. If the promoted subspace vector is not already dominated by the original MCVs of that neighbor, this subspace vector is a candidate updated MCV. Therefore, we insert it into the neighbor's affected vector set, and the propagation can then continue recursively. Next, we formally define this insertion transition relation.

DEFINITION 4. (Insertion Transition). *Given an inserted edge $e = (u, w, \ell)$, a pair $(x, \mathbf{k})$ with $\mathbf{k} \in \mathcal{A}_{\Phi}(e, x)$ and a pair $(y, \mathbf{k}')$ with $\mathbf{k}' \in \Phi_{\mathcal{G}}(y)$, let $\mathbf{r} = \mathbf{k} \wedge \mathbf{k}'$. There is an insertion transition from $(x, \mathbf{k})$ to $(y, \mathbf{r})$ of e for $x \in N_{\mathcal{G}'}(y)$ if (1) $\mathbf{k} + \mathbf{e}_{\ell} \succeq \mathbf{r} + \mathbf{e}_{\ell}$; (2) $\nexists \mathbf{h} \in \Phi_{\mathcal{G}}(x) : \mathbf{h} \succeq \mathbf{r} + \mathbf{e}_{\ell}$; and (3) $\nexists \mathbf{q} \in \Phi_{\mathcal{G}}(y) : \mathbf{q} \succeq \mathbf{r} + \mathbf{e}_{\ell}$.*

Condition (1) is always satisfied because $\mathbf{r} = \mathbf{k} \wedge \mathbf{k}'$ implies $\mathbf{k} \succeq \mathbf{r}$, and hence $\mathbf{k} + \mathbf{e}_{\ell} \succeq \mathbf{r} + \mathbf{e}_{\ell}$. We include it to explicitly indicate that the transition is triggered by the promoted vector of $\mathbf{k}$. In other words, an affected vector $\mathbf{k}$ may influence a neighbor only when its promoted vector $\mathbf{k} + \mathbf{e}_{\ell}$ newly dominates a promoted maximal subspace vector $\mathbf{r} + \mathbf{e}_{\ell}$ derived from one of the neighbor's MCV $\mathbf{k}'$.

LEMMA 2 (SAME-LEVEL INSERTION TRANSITION). *If there is an insertion transition from $(x, \mathbf{k})$ to $(y, \mathbf{r})$ for an edge insertion on layer ℓ , then $k_{\ell} = r_{\ell}$.*

By Lemma 2, after inserting an edge on layer ℓ , the affected vectors involved in insertion transitions have the same value on layer ℓ . This property enables faster condition checking for insertion transitions.

DEFINITION 5. (Insertion Affected Path). *Given an inserted edge $e = (u, w, \ell)$, two vector pairs $(x, \mathbf{k})$ and $(y, \mathbf{k}')$, an insertion affected path of e from $(x, \mathbf{k})$ to $(y, \mathbf{k}')$ is a sequence $(x_0, \mathbf{k}_0), \dots, (x_t, \mathbf{k}_t)$ such that $(x_0, \mathbf{k}_0) = (x, \mathbf{k})$, $(x_t, \mathbf{k}_t) = (y, \mathbf{k}')$, $k_{i,\ell} = k_{\ell}$ for all $0 \leq i \leq t$, and there is an insertion transition from $(x_i, \mathbf{k}_i)$ to $(x_{i+1}, \mathbf{k}_{i+1})$.*

By Definition 5, $\mathcal{A}_{\Phi}(e, \cdot)$ can be computed by collecting the affected vectors during the traverse along the insertion affected path. Next, we show how to construct upper-bound vector set $\bar{\Phi}_{\mathcal{G}'}(\cdot)$ based on $\mathcal{A}_{\Phi}(e, \cdot)$ and $\Phi_{\mathcal{G}}(\cdot)$.

LEMMA 3 (EDGE INSERTION AFFECTED SCOPE). *Let $\mathcal{G}' = \mathcal{G} \cup \{e\}$. For any vertex $v \in V$ and any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$, there exists a vector $\bar{\mathbf{k}} \in \Phi_{\mathcal{G}}(v) \cup \{\mathbf{r} + \mathbf{e}_\ell \mid \mathbf{r} \in \mathcal{A}_\Phi(e, v)\}$ such that $\bar{\mathbf{k}} \succeq \mathbf{k}'$.*

Lemma 3 indicates that, after filtering dominance relations in $\Phi_{\mathcal{G}}(v) \cup \{\mathbf{r} + \mathbf{e}_\ell \mid \mathbf{r} \in \mathcal{A}_\Phi(e, v)\}$, it is a valid upper-bound vector set $\bar{\Phi}_{\mathcal{G}'}(v)$. Therefore, after inserting an edge $e = (u, w, \ell)$, we initialize $\mathcal{A}_\Phi(e, u)$ and $\mathcal{A}_\Phi(e, w)$, and insert u and w into $\mathcal{A}(e)$ if their affected vector sets are nonempty. We then traverse the insertion affected path from the endpoints of e . Whenever an affected vector $\mathbf{r}$ of a vertex v is reached, we add v to $\mathcal{A}(e)$ and insert $\mathbf{r}$ into $\mathcal{A}_\Phi(e, v)$. Finally, we return the affected vertex set $\mathcal{A}(e)$ and the corresponding upper-bound vector set $\bar{\Phi}_{\mathcal{G}'}(v)$ for each $v \in \mathcal{A}(e)$.

5.2 Edge Deletion

In this subsection, we discuss the affected bound and affected scope under single-edge deletion. Since each dimension of an MCV must be non-negative, we use $\mathbf{k}^{(\ell, -a)} = \mathbf{k} - \min\{k_\ell, a\} \cdot \mathbf{e}_\ell$ to denote the vector obtained by decreasing the ℓ -th dimension of $\mathbf{k}$ by a , truncated at zero. In particular, $\mathbf{k}^{(\ell, -1)}$ denotes decreasing one on layer ℓ .

Affected bound. For an edge deletion, every multilayer core in the updated graph $\mathcal{G}'$ also exists in the original graph $\mathcal{G}$. Therefore, for every vertex v , each updated MCV in $\Phi_{\mathcal{G}'}(v)$ is dominated by some original MCV in $\Phi_{\mathcal{G}}(v)$. Thus, $\Phi_{\mathcal{G}}(v)$ is a natural upper-bound vector set for deletion. Next, we show that for each original MCV, decreasing one on the deleted layer gives a valid coreness vector in the updated graph.

LEMMA 4 (EDGE DELETION BOUND). *Let $\mathcal{G}' = \mathcal{G} \setminus \{e\}$, where e is deleted from layer ℓ . For any vertex $v \in V$ and any original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$, there exists an updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$ such that $\mathbf{k}' + \mathbf{e}_\ell \succeq \mathbf{k}$.*

In other words, since $\mathbf{k}'$ is an updated MCV of v , there exists a $\mathbf{k}'$ -core containing v in the updated graph. Moreover, $\mathbf{k}' + \mathbf{e}_\ell \succeq \mathbf{k}$ implies $\mathbf{k}' \succeq \mathbf{k}^{(\ell, -1)}$. Hence, the $\mathbf{k}'$ -core is contained in a $\mathbf{k}^{(\ell, -1)}$ -core, which also contains v . Therefore, $\mathbf{k}^{(\ell, -1)}$ is a valid coreness vector of v in the updated graph. If an original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$, after decreasing its value by one on the updated layer, still dominates an MCV $\mathbf{h}$ of a neighboring vertex, then there must exist an updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$ that dominates $\mathbf{h}$, i.e., $\mathbf{k}' \succeq \mathbf{k}^{(\ell, -1)} \succeq \mathbf{h}$. Therefore, the dominance relation from v to $\mathbf{h}$ is preserved in the updated graph $\mathcal{G}'$, and $\mathbf{h}$ cannot be affected by the decrease of $\mathbf{k}$. This observation of the edge deletion bound enables effective pruning during the affected scope identification. Next, we discuss how to identify the affected scope for single-edge deletion.

Affected scope. For an edge deletion $e = (u, w, \ell)$, an affected vector refers to an original MCV that may no longer remain an MCV in the updated graph. We discuss the affected vectors for endpoints of deleted edges and other vertices, respectively.

Affected vectors of deleted endpoints. At an endpoint, an original MCV can be directly affected only when the other endpoint of the deleted edge has an MCV dominating it. Thus, the affected vector sets of the endpoints of the deleted edge are initialized as $\mathcal{A}_\Phi(e, u) = \{\mathbf{k} \in \Phi_{\mathcal{G}}(u) \mid \exists \mathbf{h} \in \Phi_{\mathcal{G}}(w), \mathbf{h} \succeq \mathbf{k}\}$ and $\mathcal{A}_\Phi(e, w) = \{\mathbf{k} \in \Phi_{\mathcal{G}}(w) \mid \exists \mathbf{h} \in \Phi_{\mathcal{G}}(u), \mathbf{h} \succeq \mathbf{k}\}$, respectively.

Affected vectors of other vertices. Based on the affected vector sets of the endpoints of the deleted edge, other affected vectors can appear only when the possible loss of a dominance relation is propagated through neighboring vertices. Next, we formally define this deletion transition relation.

DEFINITION 6. (Deletion Transition). *Given a deleted edge $e = (u, w, \ell)$, a pair $(x, \mathbf{k})$ with $\mathbf{k} \in \mathcal{A}_\Phi(e, x)$ and a pair $(y, \mathbf{k}')$ with $\mathbf{k}' \in \Phi_{\mathcal{G}}(y)$, there is a deletion transition from $(x, \mathbf{k})$ to $(y, \mathbf{k}')$ of e for $x \in N_{\mathcal{G}'}(y)$ if (1) $\mathbf{k} \succeq \mathbf{k}'$ and (2) $\nexists \mathbf{h} \in \Phi_{\mathcal{G}}(x) : \mathbf{h}^{(\ell, -1)} \succeq \mathbf{k}'$.*

Specifically, when an original MCV $\mathbf{k}$ of a vertex x is affected, it may no longer dominate an MCV $\mathbf{k}'$ of a neighboring vertex y after the update, making $\mathbf{k}'$ a candidate affected vector. However, by Lemma 4, for each $\mathbf{h} \in \Phi_{\mathcal{G}}(x)$, $\mathbf{h}^{(\ell, -1)}$ remains a valid coreness vector of x in the updated graph. Thus, if there exists $\mathbf{h} \in \Phi_{\mathcal{G}}(x)$ such that $\mathbf{h}^{(\ell, -1)} \succeq \mathbf{k}'$, then $\mathbf{k}'$ can still be supported by x and cannot be affected. Therefore, such candidate vectors can be safely pruned.

LEMMA 5 (SAME-LEVEL DELETION TRANSITION). *If there is a deletion transition from $(x, \mathbf{k})$ to $(y, \mathbf{k}')$ for an edge deletion on layer ℓ , then $k_\ell = k'_\ell$.*

By Lemma 5, candidate affected vectors only need to be checked among MCVs sharing the same value on layer ℓ , which effectively reduces unnecessary exploration during the deletion transition.

DEFINITION 7. (Deletion Affected Path). *Given a deleted edge $e = (u, w, \ell)$, two vector pairs $(x, \mathbf{k})$ and $(y, \mathbf{k}')$, a deletion affected path of e from $(x, \mathbf{k})$ to $(y, \mathbf{k}')$ is a sequence $(x_0, \mathbf{k}_0), \dots, (x_t, \mathbf{k}_t)$ such that $(x_0, \mathbf{k}_0) = (x, \mathbf{k})$, $(x_t, \mathbf{k}_t) = (y, \mathbf{k}')$, $\mathbf{k}_i \in \Phi_{\mathcal{G}}(x_i)$, $k_{i, \ell} = k_\ell$ for all $0 \leq i \leq t$, and there is a deletion transition from $(x_i, \mathbf{k}_i)$ to $(x_{i+1}, \mathbf{k}_{i+1})$.*

Therefore, $\mathcal{A}_\Phi(e, v)$ can be computed by collecting the affected vectors $\mathbf{k}$ along the deletion affected path of e . The following lemma ensures that it contains all original MCVs of v that may need to be updated after the deletion.

LEMMA 6 (EDGE DELETION AFFECTED SCOPE). *Let $\mathcal{G}' = \mathcal{G} \setminus \{e\}$. For any vertex $v \in V$, if an original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ may become invalid after deleting e , then $\mathbf{k} \in \mathcal{A}_\Phi(e, v)$.*

Since every updated MCV in $\Phi_{\mathcal{G}'}(v)$ is dominated by some original MCV in $\Phi_{\mathcal{G}}(v)$, the original MCV set is an upper-bound vector set. Therefore, after deleting an edge $e = (u, w, \ell)$, we initialize $\mathcal{A}_\Phi(e, u)$ and $\mathcal{A}_\Phi(e, w)$ as the affected vector sets of the endpoints of the deleted edge, and insert the endpoints into $\mathcal{A}(e)$ if their affected vector sets are nonempty. We then traverse deletion affected paths of the deleted edge e . Whenever an affected vector $\mathbf{k}$ of a vertex v is reached, we add v to $\mathcal{A}(e)$ and insert $\mathbf{k}$ into $\mathcal{A}_\Phi(e, v)$. Finally, we return $\mathcal{A}(e)$ and the upper-bound vector set $\bar{\Phi}_{\mathcal{G}'}(\cdot) = \Phi_{\mathcal{G}}(\cdot)$.

5.3 Identify Affected Scope

Based on the above discussion, we use a unified procedure to identify the affected scope for edge insertion and deletion. Algorithm 2 shows the pseudocode. It initializes the upper-bound vector sets (line 1), and then computes the affected vector sets of the endpoints of the updated edge (line 2). According to the operation type, it traverses either insertion affected paths or deletion affected paths of e to obtain $\mathcal{A}_\Phi(e, \cdot)$ and $\mathcal{A}(e)$ (lines 3-6). For insertion, every affected vector $\mathbf{k} \in \mathcal{A}_\Phi(e, v)$ is promoted to $\mathbf{k} + \mathbf{e}_\ell$ and inserted into

Algorithm 2: Identify Scope (IS)

Input: Updated graph $\mathcal{G}'$; input MCV sets $\Phi_{\mathcal{G}}(\cdot)$; updated edge $e = (u, w, \ell)$; operation $op \in \{\text{insert}, \text{delete}\}$
Output: Affected vertex set $\mathcal{A}(e)$ and upper-bound vector sets $\bar{\Phi}_{\mathcal{G}'}(\cdot)$

```
1  $\bar{\Phi}_{\mathcal{G}'}(\cdot) \leftarrow \Phi_{\mathcal{G}}(\cdot)$ ;
2 Compute  $\mathcal{A}_{\Phi}(e, u)$  and  $\mathcal{A}_{\Phi}(e, w)$ ;
3 if  $op = \text{insert}$  then
4    $(\mathcal{A}_{\Phi}(e, \cdot), \mathcal{A}(e)) \leftarrow \text{traverse insertion affected paths of } e$ ;
5 else
6    $(\mathcal{A}_{\Phi}(e, \cdot), \mathcal{A}(e)) \leftarrow \text{traverse deletion affected paths of } e$ ;
7 if  $op = \text{insert}$  then
8   foreach  $v \in \mathcal{A}(e)$  and  $\mathbf{k} \in \mathcal{A}_{\Phi}(e, v)$  do
9     insert  $\mathbf{k} + \mathbf{e}_{\ell}$  into  $\bar{\Phi}_{\mathcal{G}'}(v)$ ;
10    Filter dominance relations in  $\bar{\Phi}_{\mathcal{G}'}(v)$ ;
11 return  $(\mathcal{A}(e), \bar{\Phi}_{\mathcal{G}'}(\cdot))$ ;
```

the upper-bound vector set of v , followed by dominance filtering (lines 7-10). For deletion, the upper-bound vector sets remain the original MCV sets as initialized in line 1. Finally, the algorithm returns $\mathcal{A}(e)$ and $\bar{\Phi}_{\mathcal{G}'}(\cdot)$ (line 11).

Complexity analysis. Let $P = \max_{v \in V} |\Phi_{\mathcal{G}}(v)|$, $\lambda = |L|$, and W_e be the number of candidate adjacent vector pairs checked during the affected path traversal triggered by e . The dominant cost arises from traversing the affected path. For each candidate transition, dominance is checked against at most a constant number of MCV sets of size at most P , yielding a cost of $O(P\lambda)$ per transition. Therefore, the total time cost of Algorithm 2 is $O(W_e P \lambda)$.

6 LOCAL CONVERGENCE

In this section, we first provide an overview of the local convergence procedure, followed by a naive implementation. We then present several optimization techniques to improve its efficiency.

6.1 Local Convergence Overview

Given a multilayer graph $\mathcal{G}$ and its upper-bound vector sets $\bar{\Phi}_{\mathcal{G}}(\cdot)$, the local convergence procedure computes MCV sets $\Phi_{\mathcal{G}}(\cdot)$ from $\bar{\Phi}_{\mathcal{G}}(\cdot)$ by repeatedly refining invalid upper-bound vectors until all remaining vectors are valid. Therefore, once we have identified the upper-bound vector sets $\bar{\Phi}_{\mathcal{G}'}(\cdot)$ of the updated graph $\mathcal{G}'$, we can leverage the local convergence procedure to compute $\Phi_{\mathcal{G}'}(\cdot)$. It consists of two key operations: validation and refinement.

Validation rule. Recall from Definition 3 that if $\bar{\mathbf{k}} \in \bar{\Phi}_{\mathcal{G}}(v)$ is an MCV of v , then v must have enough neighbors containing MCVs that dominate $\bar{\mathbf{k}}$. We say that $\bar{\mathbf{k}} = [\bar{k}_{\ell}]_{\ell \in L}$ is *valid* if for every layer $\ell \in L$, there exist at least $\bar{k}_{\ell}$ neighbors u of v on layer ℓ such that $\exists \bar{\mathbf{k}}' \in \bar{\Phi}_{\mathcal{G}}(u)$ with $\bar{\mathbf{k}}' \succeq \bar{\mathbf{k}}$.

LEMMA 7 (VALIDATION RULE). *If every vector $\bar{\mathbf{k}} \in \bar{\Phi}_{\mathcal{G}}(v)$ is valid for any $v \in V$, then $\bar{\mathbf{k}}$ is a coreness vector of v in $\mathcal{G}$.*

By Lemma 7, we can check the upper-bound vectors that fail to be an MCV. For each invalid upper-bound vector, we follow a refinement rule to relax its constraint by decreasing one of its entries to explore a possible MCV.

Algorithm 3: Local Convergence (LC)

Input: A multilayer graph $\mathcal{G}$; upper-bound vector sets $\bar{\Phi}_{\mathcal{G}}(\cdot)$; affected vertex set $\mathcal{A}(e)$
Output: The MCV sets $\Phi_{\mathcal{G}}(\cdot)$

```
1 Initialize a unique queue  $Q \leftarrow \mathcal{A}(e)$ ;
2 while  $Q \neq \emptyset$  do
3   Dequeue a vertex  $v$  from  $Q$ ;
4    $changed \leftarrow \text{false}$ ;
5   while  $\exists \bar{\mathbf{k}} \in \bar{\Phi}_{\mathcal{G}}(v)$  such that  $\bar{\mathbf{k}}$  is invalid do
6     Remove  $\bar{\mathbf{k}}$  from  $\bar{\Phi}_{\mathcal{G}}(v)$ ;
7     Insert a safe refinement set  $\mathcal{R}(\bar{\mathbf{k}})$  into  $\bar{\Phi}_{\mathcal{G}}(v)$ ;
8     Filter dominance relations in  $\bar{\Phi}_{\mathcal{G}}(v)$ ;
9      $changed \leftarrow \text{true}$ ;
10  if  $changed = \text{true}$  then
11    Enqueue all vertices in  $N_{\mathcal{G}}(v) \cap \mathcal{A}(e)$  into  $Q$ ;
12 return  $\Phi_{\mathcal{G}}(\cdot) \leftarrow \bar{\Phi}_{\mathcal{G}}(\cdot)$ ;
```

Refinement rule. If an upper-bound vector $\bar{\mathbf{k}}$ is invalid, we remove it and replace it with a set of safe refinement vectors $\mathcal{R}(\bar{\mathbf{k}})$. The refinement is *safe* if it satisfies the following two conditions: (1) every $\bar{\mathbf{k}}' \in \mathcal{R}(\bar{\mathbf{k}})$ is strictly dominated by $\bar{\mathbf{k}}$, i.e., $\bar{\mathbf{k}} \succ \bar{\mathbf{k}}'$; (2) for every MCV $\mathbf{k}^* \in \Phi_{\mathcal{G}}(v)$ with $\bar{\mathbf{k}} \succeq \mathbf{k}^*$, if $\bar{\mathbf{k}} \neq \mathbf{k}^*$, then there exists $\bar{\mathbf{k}}' \in \mathcal{R}(\bar{\mathbf{k}})$ such that $\bar{\mathbf{k}}' \succeq \mathbf{k}^*$.

LEMMA 8 (SAFE REFINEMENT). *Suppose that for every vertex $x \in V$, every MCV of x is dominated by at least one vector in $\bar{\Phi}_{\mathcal{G}}(x)$. Given an invalid vector $\bar{\mathbf{k}} \in \bar{\Phi}_{\mathcal{G}}(v)$ and a safe refinement set $\mathcal{R}(\bar{\mathbf{k}})$, let $\bar{\Phi}'_{\mathcal{G}}(v) = \bar{\Phi}_{\mathcal{G}}(v) \setminus \{\bar{\mathbf{k}}\} \cup \mathcal{R}(\bar{\mathbf{k}})$. Then every MCV of v is dominated by at least one vector in $\bar{\Phi}'_{\mathcal{G}}(v)$.*

Intuitively, the validation rule removes vectors in $\bar{\Phi}_{\mathcal{G}}(v)$ that fail to be an MCV, while the refinement rule ensures the maximality of the retained vectors. As a result, $\bar{\Phi}_{\mathcal{G}}(v)$ iteratively converges to $\Phi_{\mathcal{G}}(v)$. Specifically, the local convergence procedure is shown in Algorithm 3. It first initializes a unique queue with affected vertices in $\mathcal{A}(e)$ (line 1). For each dequeued vertex (line 3), it repeatedly checks whether there exists a vector in $\bar{\Phi}_{\mathcal{G}}(v)$ that is invalid (line 5). Each invalid vector is removed and replaced with a safe refinement set (lines 6-7). Then, dominance filtering is applied to maintain the mutual incomparability of vectors in $\bar{\Phi}_{\mathcal{G}}(v)$ (line 8). If $\bar{\Phi}_{\mathcal{G}}(v)$ changes, only affected neighbors of v are re-enqueued for further validation (lines 10-11). The procedure terminates when the queue becomes empty and returns the remaining upper-bound sets as MCV sets (line 12). Since the validation and refinement of each vertex only depend on the upper-bound vector sets of its neighbors, multiple affected vertices can be processed in parallel based on the current neighbor states. Whenever a neighboring upper-bound vector set changes, the corresponding affected vertices are re-enqueued for reprocessing, ensuring that all dependency changes are correctly propagated. Due to the page limit, the correctness analysis of Algorithm 3 can be found in the appendix of our technical report [59].

6.2 Naive Implementation

In this subsection, we introduce a naive implementation of validation and refinement in Algorithm 3.

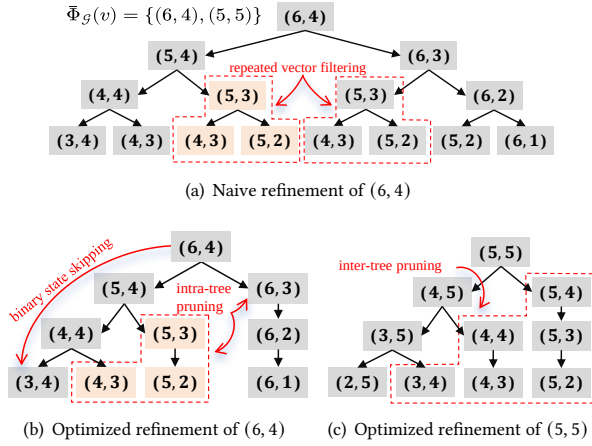


Figure 2: An example of refinement

Naive validation. Given a vertex v and a candidate vector $\bar{k} \in \bar{\Phi}_{\mathcal{G}}(v)$, the naive validation scans $\bar{\Phi}_{\mathcal{G}}(u)$ for each neighbor $u \in N_{\mathcal{G}}(v)$ and checks whether there are enough neighbors in each layer that contain upper-bound vectors dominating $\bar{k}$.

Naive refinement. For refinement, the naive implementation uses a simple splitting strategy: an invalid vector $\bar{k}$ is split by decreasing one dimension by one, generating $\{\bar{k}^{(t,-1)}\}_{t \in L}$, where $\bar{k}^{(t,-1)} = \bar{k} - \min\{\bar{k}_t, 1\} \cdot \mathbf{e}_t$. Suppose that $\mathbf{k}^*$ is an MCV dominated by $\bar{k}$. Since $\bar{k}$ is invalid, $\bar{k}$ cannot be an MCV. Therefore, $\bar{k} \succ \mathbf{k}^*$. This refinement is safe because at least one dimension of $\bar{k}$ is larger than $\mathbf{k}^*$, and decreasing that dimension by one still yields a vector dominating $\mathbf{k}^*$.

EXAMPLE 1. Figure 2(a) is an example of naive refinement. Starting from an invalid upper-bound vector $\bar{k} = (6, 4) \in \bar{\Phi}_{\mathcal{G}}(v)$, the naive refinement generates candidate vectors by decreasing each dimension of $\bar{k}$ by one. For example, $(6, 4)$ is split into $(5, 4)$ and $(6, 3)$, which are then recursively validated until maximal valid vectors are identified.

6.3 Optimized Implementation

Despite its simplicity, the naive implementation incurs significant overhead for two main reasons. First, naive validation repeatedly scans upper-bound vectors of neighbors. Second, naive refinement may generate a large number of invalid intermediate vectors. To address these limitations, we introduce a series of optimizations to accelerate the validation and refinement phase, respectively.

6.3.1 Validation optimization. To reduce redundant accesses to neighboring vertices' upper-bound vectors, we introduce a reuse strategy to leverage historical validation results efficiently.

OPT-1: Reuse strategy. The reuse strategy consists of two operations: *recording* and *updating*. The former records historical validation results of neighbors, while the latter incrementally maintains the recorded information.

Recording. For each upper-bound vector $\bar{k} \in \bar{\Phi}_{\mathcal{G}}(v)$ and each neighbor $u \in N_{\mathcal{G}}(v)$, we classify the upper-bound vectors in $\bar{\Phi}_{\mathcal{G}}(u)$ into three categories: the dominant set $Dom(\bar{k}, v, u)$, the fail set $Fail(\bar{k}, v, u)$, and the candidate set $Cand(\bar{k}, v, u)$. For any vector

$\bar{k}' \in \bar{\Phi}_{\mathcal{G}}(u)$, if it has been validated, we place it into $Dom(\bar{k}, v, u)$ when $\bar{k}' \succeq \bar{k}$, and into $Fail(\bar{k}, v, u)$ otherwise. If $\bar{k}'$ has not yet been validated, we store it in $Cand(\bar{k}, v, u)$ for future checking.

When validating $\bar{k}$, we first count the neighbors u whose dominant set $Dom(\bar{k}, v, u)$ is non-empty. If enough neighbors are found, $\bar{k}$ is valid without further accesses to neighbors' upper-bound vectors. Otherwise, we only need to examine the neighbors with empty dominant sets. For these neighbors, vectors in $Fail(\bar{k}, v, u)$ can be safely skipped, since they have already been validated and cannot dominate $\bar{k}$. Thus, we only check the vectors in $Cand(\bar{k}, v, u)$ to determine whether any of them can dominate $\bar{k}$, and stop once enough dominating neighbors are found.

Updating. The updating operation aims to update the reuse sets incrementally when $\bar{\Phi}_{\mathcal{G}}(v)$ changes for the refinement, which is based on the following inheritance property.

LEMMA 9 (DOMINANCE INHERITANCE). Consider an upper-bound vector of v refined from $\bar{k}$ to $\bar{k}^{new}$. For any neighbor u of v , every $\bar{k}' \in Dom(\bar{k}, v, u)$ satisfies $\bar{k}' \succ \bar{k}^{new}$.

By Lemma 9, we incrementally maintain the affected reuse sets. If $\bar{k}$ is invalid, then for any vector $\bar{k}^{new}$ in the refinement set of $\bar{k}$, we directly inherit $Dom(\bar{k}, v, u)$ as $Dom(\bar{k}^{new}, v, u)$, while moving $Fail(\bar{k}, v, u)$ to $Cand(\bar{k}^{new}, v, u)$. This is because $\bar{k} \succ \bar{k}^{new}$ by refinement rule, and vectors that fail to dominate $\bar{k}$ may dominate $\bar{k}^{new}$. Then, we update the reuse sets of affected neighbors that contain $\bar{k}$. For affected neighbor u of v with $\bar{k}' \in \bar{\Phi}_{\mathcal{G}}(u)$, $\bar{k}^{new}$ may fail to dominate $\bar{k}'$. Therefore, $\bar{k}$ is moved from $Dom(\bar{k}', u, v)$ and $\bar{k}^{new}$ is added to $Cand(\bar{k}', u, v)$ for validation.

6.3.2 Refinement Optimization. The refinement phase aims to refine invalid upper-bound vectors into maximal valid ones. The naive refinement decreases each dimension by one at each step, forming an $|L|$ -ary splitting tree. However, this process may generate a large number of redundant intermediate vectors. To improve efficiency, we optimize the refinement phase from three aspects: repeated vector filtering, binary state skipping, and redundant vector pruning.

OPT-2: Repeated vector filtering. In the splitting tree of naive refinement, the same vector can be generated through different sequences of dimension reductions, resulting in redundant exploration. As illustrated in Figure 2(a), the vector $(5, 3)$ is generated from both the vectors $(5, 4)$ and $(6, 3)$. Such repeated vectors cause redundant validation and refinement. To address this problem, we introduce an ordered splitting rule that prevents repeated vectors from being generated.

Ordered splitting rule. For each splitting vector, we maintain an index $idx(\bar{k})$ indicating the last decreased layer. The initial vector has $idx(\bar{k}) = 1$. When splitting a vector with index i , we only allow decreases on layers $j \geq i$. By decreasing the j -th dimension of $\bar{k}$, each generated child records the index as j . In this way, every root-to-leaf path decreases layers in a non-decreasing order.

LEMMA 10 (REPEATED VECTOR FILTERING). The ordered splitting rule generates each vector at most once.

EXAMPLE 2. By applying the ordered splitting rule, the splitting tree shown in Figure 2(b) avoids the repeated vectors such as $(5, 3)$ that appeared in Figure 2(a).

OPT-3: Binary state skipping. For an invalid upper-bound vector, decreasing one dimension by one may generate a long sequence of invalid vectors before reaching the maximal valid vector. However, we observe that once a vector becomes valid, all vectors dominated by this vector are also valid. In contrast, if a vector is invalid, all vectors dominating it cannot be valid.

LEMMA 11 (VALIDITY HIERARCHY). Consider two upper-bound vectors $\bar{k}$ and $\bar{k}'$ of the same vertex, where $\bar{k} \succeq \bar{k}'$. If $\bar{k}$ is valid, $\bar{k}'$ is valid. If $\bar{k}'$ is invalid, $\bar{k}$ is invalid.

Based on Lemma 11, we exploit the hierarchy of vector validity to perform binary search on the splitting tree, thereby efficiently identifying the maximal valid vector. When splitting a vector on layer ℓ , instead of directly setting $\bar{k}_\ell$ to $\bar{k}_\ell - 1$, we search within an interval $[low_\ell, \bar{k}_\ell - 1]$ for the largest value on layer ℓ that yields a valid vector while keeping the other dimensions unchanged. The lower boundary low_ℓ is set to 0 initially. At each step, we test the midpoint vector by validation rule: if it is valid, we move the lower boundary upward; otherwise, we move the upper boundary downward. Thus, binary state skipping reduces the depth of each one-dimensional descent from linear in the value range to logarithmic.

EXAMPLE 3. In Figure 2(b), suppose that $(3, 4)$ is the maximal valid vector along the first dimension. Therefore, any vector with a larger first-layer value, such as $(4, 4)$ and $(5, 4)$, must be invalid. Under the naive refinement strategy, these intermediate vectors are validated sequentially before reaching $(3, 4)$. In contrast, by applying binary search on the first dimension, we can directly validate $(3, 4)$ first, thereby avoiding unnecessary validation on $(5, 4)$ and $(4, 4)$.

OPT-4: Redundant vector pruning. The redundant vector pruning strategy is designed for two scenarios: intra-tree pruning and inter-tree pruning.

Intra-tree pruning. The key idea of the intra-tree pruning is based on the observation that once a vector is dominated by a discovered valid vector of the same vertex, neither this vector nor any of its descendants can become maximal. Therefore, we maintain a maximal valid vector set $Val(v)$ for each vertex to prune these vectors. To fully exploit the maximal valid vector set $Val(v)$, maximal valid vectors should be discovered as early as possible. To this end, we propose a right-subtree-first traversal strategy.

LEMMA 12 (SUBTREE DOMINANCE). Consider a vector $\bar{k}$ in a splitting tree of the refinement phase and two child subtrees T_i and T_j generated by decreasing the value of $\bar{k}$ on layers i and j with $i < j$, respectively. Then no vector in T_i dominates any vector in T_j . In contrast, a vector in T_j may dominate a vector in T_i .

By Lemma 12, we traverse right subtrees first so that maximal valid vectors can be discovered earlier to prune the subsequent dominating vectors.

EXAMPLE 4. As illustrated in Figure 2(b), we first explore the right subtree of $(6, 4)$. After identifying $(6, 3)$ as a maximal valid vector, we insert it into $Val(v)$. Subsequently, when traversing the left subtree, we

reach $(5, 3)$. Since $(6, 3) \succeq (5, 3)$, $(5, 3)$ is dominated by an existing maximal valid vector. Moreover, every descendant of $(5, 3)$ is also dominated by $(6, 3)$. Hence, none of them can become maximal valid vectors, and the subtree rooted at $(5, 3)$ can be pruned safely.

Inter-tree pruning. A vertex may have multiple invalid vectors, whose split descendant vectors are validated against the same neighbors. Although these invalid vectors are mutually incomparable, their descendant subspaces may overlap. For a root vector $\bar{r} \in \mathbb{N}^{|L|}$, let $\mathcal{D}(\bar{r}) = \{\mathbf{x} \in \mathbb{N}^{|L|} \mid \bar{r} \succeq \mathbf{x}\}$ denote its subspace vector set.

LEMMA 13 (SUBSPACE COVERAGE). If the splitting tree rooted at $\bar{r}$ has been fully explored, then no vector in $\mathcal{D}(\bar{r})$ can produce a new maximal valid vector.

EXAMPLE 5. In Figure 2, suppose that $\bar{\Phi}_{\mathcal{G}}(v) = \{(6, 4), (5, 5)\}$, where both vectors are invalid upper-bound vectors. After completely refining $(6, 4)$, all maximal valid vectors contained in $\mathcal{D}((6, 4))$ have been identified. Subsequently, when refining $(5, 5)$, as shown in Figure 2(c), the vector $(5, 4)$ is generated. Since $(5, 4) \in \mathcal{D}((6, 4))$, any maximal valid vector reachable from the subtree rooted at $(5, 4)$ has already been discovered during the refinement of $(6, 4)$ or dominated by a previously discovered maximal valid vector. Hence, the subtree rooted at $(5, 4)$ can be safely pruned.

Based on Lemma 13, we maintain a processed-root set $Root(v)$ for each vertex v . Once all maximal valid vectors derived from an invalid root vector of the splitting tree have been identified, we insert this root vector into $Root(v)$. We use it to restrict the binary search range in OPT-3, so that vectors covered by previously processed roots can be pruned. Specifically, suppose we split $\bar{k}$ on layer ℓ and test vectors obtained by replacing $\bar{k}_\ell$ with x while keeping the other dimensions unchanged. For any $\bar{r} \in Root(v)$ satisfying $\bar{r}_j \geq \bar{k}_j$ for every layer $j \neq \ell$, all tested vectors with $x \leq \bar{r}_\ell$ fall into $\mathcal{D}(\bar{r})$. Therefore, before binary search on layer ℓ , we set $low_\ell \leftarrow \max\{low_\ell, \bar{r}_\ell + 1\}$ over all such roots. If $low_\ell > \bar{k}_\ell - 1$, this splitting dimension has no uncovered vector left; otherwise, binary search is performed only on $[low_\ell, \bar{k}_\ell - 1]$.

Complexity analysis. Let $\bar{P} = \max_{v \in V} |\bar{\Phi}_{\mathcal{G}}(v)|$, $\lambda = |L|$, $d_v = |N_{\mathcal{G}}(v)|$, and $K_v = \max_{\bar{k} \in \bar{\Phi}_{\mathcal{G}}(v), \ell \in L} (\bar{k}_\ell + 1)$. Let $\mathcal{F}_v^{\text{opt}}$ denote the number of vectors explored during the refinement phase for vertex v after applying repeated vector filtering, binary state skipping, and redundant vector pruning. Since binary state skipping reduces the search in each dimension from testing $O(K_v)$ values to $O(\log K_v)$ values, the resulting search space for vertex v is bounded by $O(\bar{P}(\log K_v)^\lambda)$. Let $\mathcal{I}_v$ denote the number of identifications on neighbors' upper-bound vectors performed during a single validation under the reuse strategy. Each visited vector incurs a validation cost of $O(\mathcal{I}_v \lambda)$. Therefore, Algorithm 3 with the optimized implementation takes $O\left(\sum_{v \in \mathcal{A}(e)} \mathcal{F}_v^{\text{opt}} \mathcal{I}_v \lambda\right)$ time, where $\mathcal{I}_v \leq d_v \bar{P}$ and $\mathcal{F}_v^{\text{opt}} \leq \bar{P}(\log K_v)^\lambda$.

7 EXTENSION TO BATCH UPDATES

In this section, we extend our multilayer core maintenance algorithm to support batch edge updates ΔE . The key idea is to partition ΔE into edge groups such that updating all edges in the same group simultaneously still preserves the single-edge update bound of

Lemma 1 and Lemma 4. Therefore, the edges within each group can be processed simultaneously by merging their affected scopes. Next, we first introduce a hybrid edge grouping strategy and then propose our multilayer core maintenance algorithm for batch updates.

7.1 Hybrid Edge Grouping Strategy

In this subsection, we introduce our hybrid edge grouping strategy based on two types of edge groups, i.e., *disjoint edge group* and *joint edge group*.

Disjoint edge group. Let updated edge set $\Delta E = \{\Delta E_\ell\}_{\ell \in L}$, where ΔE_ℓ is the updated edges on layer ℓ . An edge set $B \subseteq \Delta E_\ell$ is called a disjoint edge group if no two edges in B share an endpoint. Intuitively, after updating all edges in B , each vertex in a k -core gains or loses at most one incident edge on layer ℓ . Based on this observation, we have the following lemma.

LEMMA 14 (DISJOINT EDGE GROUP BOUND). *Let $B \subseteq \Delta E_\ell$ be a disjoint edge group updated on layer ℓ . If B is inserted, then for any vertex $v \in V$ and any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G} \cup B}(v)$, there exists an original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ such that $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$. If B is deleted, then for any vertex $v \in V$ and any original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$, there exists an updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G} \setminus B}(v)$ such that $\mathbf{k}' + \mathbf{e}_\ell \succeq \mathbf{k}$.*

By Lemma 14, updating all edges in a disjoint edge group simultaneously preserves the single-edge update bound. Therefore, the entire group can be processed as a single update by merging the affected scope of each edge.

Joint edge group. Despite the effectiveness of disjoint edge groups, they may degenerate when a large number of updated edges share a common endpoint. To address this issue, we introduce a concept of the joint edge group to process these edges simultaneously. Let $\mu_\ell(v) = \min_{\mathbf{k} \in \Phi_{\mathcal{G}}(v)} k_\ell$ and $\eta_\ell(v) = \max_{\mathbf{k} \in \Phi_{\mathcal{G}}(v)} k_\ell$ as the minimum and maximum layer- ℓ values among the original MCVs of v , respectively. Formally, we define the joint edge group as follows.

DEFINITION 8. (Joint Edge Group). *Let $B \subseteq \Delta E_\ell$ be a set of updated edges on layer ℓ and C_B be the set of common endpoints in B . B is a joint edge group if (1) no edge in B connects two vertices in C_B ; and (2) for every edge $(u, w, \ell) \in B$ with an endpoint in C_B , $\mu_\ell(u) > \eta_\ell(w)$ if $u \in C_B$, and $\mu_\ell(w) > \eta_\ell(u)$ if $w \in C_B$.*

Next, we show that the update of edges in each joint edge group still preserves the single-edge update bound.

LEMMA 15 (JOINT EDGE GROUP BOUND). *Let $B \subseteq \Delta E_\ell$ be a joint edge group. If B is inserted, then for any vertex $v \in V$ and any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G} \cup B}(v)$, there exists an original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ such that $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$. If B is deleted, then for any vertex $v \in V$ and any original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$, there exists an updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G} \setminus B}(v)$ such that $\mathbf{k}' + \mathbf{e}_\ell \succeq \mathbf{k}$.*

Intuitively, if the value of every MCV of the common endpoint is greater than those of the non-common endpoints on the updated layer, then this common endpoint cannot be affected according to the same-level requirement of Lemma 2 and Lemma 5. Moreover, each non-common endpoint is incident to at most one updated edge on layer ℓ in B . Therefore, updating all edges in the joint edge group satisfies the single-edge update bound. By combining disjoint and joint edge groups, our hybrid grouping strategy greedily constructs edge groups from ΔE_ℓ . Specifically, for each selected edge, we insert

Algorithm 4: Multilayer Core Maintenance (MCM)

Input: A multilayer graph $\mathcal{G}$; input MCV sets $\Phi_{\mathcal{G}}(\cdot)$; updated edge set $\Delta E = \{\Delta E_\ell\}_{\ell \in L}$; operation $op \in \{\text{insert}, \text{delete}\}$
Output: Updated MCV sets $\Phi_{\mathcal{G}'}(\cdot)$

```

1  $\mathcal{G}_0 \leftarrow \mathcal{G}$ ;  $\Phi_{\mathcal{G}_0}(\cdot) \leftarrow \Phi_{\mathcal{G}}(\cdot)$ ;  $t \leftarrow 0$ ;
2 foreach  $\Delta E_\ell \in \Delta E$  do
3    $R \leftarrow \Delta E_\ell$ ;
4   while  $R \neq \emptyset$  do
5      $B \leftarrow$  apply hybrid grouping strategy on  $R$ ;  $R \leftarrow R \setminus B$ ;
6      $\mathcal{G}_{t+1} \leftarrow \mathcal{G}_t \cup B$  if  $op = \text{insert}$ ; otherwise  $\mathcal{G}_{t+1} \leftarrow \mathcal{G}_t \setminus B$ ;
7      $(\mathcal{A}(B), \bar{\Phi}_{\mathcal{G}_{t+1}}(\cdot)) \leftarrow$  Identify affected scope of  $B$ ;
8     Filter dominance relations in  $\bar{\Phi}_{\mathcal{G}_{t+1}}(\cdot)$ ;
9      $\Phi_{\mathcal{G}_{t+1}}(\cdot) \leftarrow \text{LocalConvergence}(\mathcal{G}_{t+1}, \bar{\Phi}_{\mathcal{G}_{t+1}}(\cdot), \mathcal{A}(B))$ ;
      $t \leftarrow t + 1$ ;
10 return  $\Phi_{\mathcal{G}'}(\cdot) \leftarrow \Phi_{\mathcal{G}_t}(\cdot)$ ;
```

it into an existing group if either it shares no endpoint with the edges already in that group, or the resulting enlarged group still satisfies the conditions of the joint edge group.

7.2 Batch Updates

Based on the hybrid grouping strategy, we propose the multilayer core maintenance algorithm for batch-edge updates.

Batch affected scope. For an edge group B constructed by the hybrid grouping strategy, Lemmas 14 and 15 show that updating B preserves the single-edge update bound. Therefore, after applying B to the graph, we can traverse the affected paths rooted at each edge in B on the updated graph. Different traversals may discover overlapping affected vectors, but the bound provided by Lemmas 14 and 15 prevents such overlaps from accumulating into a larger vector change. Finally, after merging the affected scopes and candidate upper-bound vectors, we can leverage Algorithm 3 to refine these upper-bound vectors into the updated MCVs.

Algorithm 4 shows the pseudocode of our multilayer core maintenance algorithm for batch-edge updates. The algorithm initializes the graph and MCV sets (line 1), and processes the updated edges layer by layer (line 2). For each layer, it repeatedly scans the remaining edges and greedily constructs a group B based on the hybrid grouping strategy (line 5). Then, the algorithm constructs the updated graph with B (line 6), and identifies the affected scope of updated edges (line 7). Next, it filters dominated upper-bound vectors (line 8) and invokes Algorithm 3 to refine the MCV sets (line 9). The process terminates after all updated edges have been processed and returns the updated MCV sets (line 10).

Complexity analysis. Let $m_\ell = |\Delta E_\ell|$, $\lambda = |L|$, $P = \max_{v \in V} |\Phi_{\mathcal{G}}(v)|$, g_ℓ be the number of groups constructed on layer ℓ , and g be the total number of groups of ΔE . Since Algorithm 4 scans all remaining edges on a layer in each iteration, the total grouping scan cost is $O(\sum_{\ell \in L} (g_\ell m_\ell))$. Hence, computing $\mu_\ell(\cdot)$ and $\eta_\ell(\cdot)$ for updated edges costs $O(\sum_{\ell \in L} (g_\ell m_\ell P))$. For an edge e , let W_e be the number of adjacent vector pairs checked during the identification of affected scope. As analyzed in Algorithm 2, the affected scope identification cost over all edges is $O(\sum_{e \in \Delta E} (W_e P \lambda))$. The cost of local convergence has been analyzed in Algorithm 3, and we denote it by $T_{\text{conv}}(B_i)$, where B_i is the i -th group. Therefore, Algorithm 4 takes $O(\sum_{\ell \in L} (g_\ell m_\ell P) + \sum_{e \in \Delta E} (W_e P \lambda) + \sum_{i=1}^g T_{\text{conv}}(B_i))$ time.

Table 1: Dataset statistics

Dataset	V	E	L	$\min_{\ell \in L} E_\ell $	$\max_{\ell \in L} E_\ell $
Askubuntu(AU) [1]	159,316	461,248	5	13	147,120
Superuser(SU) [49]	194,085	730,988	6	37,305	158,762
Yago(YG) [10]	123,183	1,072,765	37	17	377,139
Obama(OB) [40]	2,279,535	3,827,964	3	557,445	1,807,134
Pope(PO) [42]	2,064,866	5,492,251	3	423,103	3,092,745
Boston(BT) [42]	4,374,479	9,130,543	3	665,890	5,909,061

8 EXPERIMENTS

In this section, we conduct experiments to evaluate our maintenance algorithms on real-world multilayer graphs.

8.1 Experimental Setup

Datasets. We use six real-world multilayer graph datasets in our experiments. Table 1 shows their statistics. Specifically, *Askubuntu* and *Superuser* are online question-answering interaction networks, with layers representing interactions in different time intervals. *Yago* is a knowledge graph whose layers correspond to different relation types. *Obama*, *Pope*, and *Boston* are social networks, with layers corresponding to retweet, mention, and reply relationships. **Competitors.** We compare several algorithms in our experiments:

- **DEC** [15]: the classic multilayer core decomposition algorithm.
- **PARDEC** [35]: the state-of-the-art multilayer core decomposition algorithm.
- **MCM**: our multilayer core maintenance algorithm.
- **IS, LC**: the affected scope identification and local convergence procedures of MCM, respectively.
- **MCM-S**: a batch-update variant that processes updated edges sequentially by applying MCM to each edge individually.

Setup. The algorithms are implemented in C++ with -O3 compiler optimization and OpenMP-based parallelization. Experiments are conducted on a Linux machine equipped with an Intel Xeon Gold 5520 2.2 GHz processor (56 physical cores with two-way hyper-threading) and 125 GB RAM. Following [7], we randomly generate 500 single-edge updates. For batch-edge updates, we generate 500 update batches, each containing 500 updated edges. We report the average running time per update in milliseconds (ms). By default, we conduct the experiments under the single-edge update, and the number of threads is set to 32. Due to space limitations, results on some datasets can be found in the appendix of [59].

8.2 Experimental Results

Exp-1: Overall performance comparison. We compare MCM against DEC and PARDEC under the default experimental settings. Table 2 reports the average update time for edge insertions and deletions. We break down the overall runtime of MCM into IS and LC. The results show that MCM consistently achieves the best performance on all datasets and for both update operations. Compared with the state-of-the-art method PARDEC, MCM achieves speedups ranging from $\times 28$ to $\times 5,576$ for insertions and from $\times 138$ to $\times 1,881$ for deletions. These results demonstrate that decomposition-based methods are inefficient for dynamic updates, whereas MCM can maintain MCVs efficiently by restricting computation to the affected scope. We also observe that edge insertions generally incur higher

Table 2: Overall performance comparison (ms)

Operation	Dataset	DEC	PARDEC	MCM	Speedup	IS	LC
Insertion	AU	11,115	89	2,629	$\times 34$	2,260	0.369
	SU	102,263	643	22,815	$\times 28$	21,219	1.595
	YG	27,869	2,704	0.485	$\times 5,576$	0.297	0.188
	OB	12,874	1,763	2.982	$\times 591$	2,044	0.938
	PO	62,330	611	3.527	$\times 173$	2,249	1.277
	BT	194,159	7,190	9.732	$\times 739$	7,265	2.467
Deletion	AU	11,114	89	0.477	$\times 186$	0.386	0.092
	SU	102,213	645	4.661	$\times 138$	3,959	0.703
	YG	27,815	2,727	1.450	$\times 1,881$	0.910	0.539
	OB	12,672	1,758	3.039	$\times 579$	2,018	1.021
	PO	62,312	618	2.397	$\times 258$	1,614	0.784
	BT	194,178	7,117	5.581	$\times 1,275$	3,708	1.873

Table 3: Effect of optimizations (ms)

Operation	Dataset	FULL	-OPT1	-OPT2	-OPT3	-OPT4
Insertion	AU	0.369	0.355	0.362	0.376	0.561
	SU	1.595	1.589	2.581	1.604	2.294
	YG	0.188	0.190	0.953	0.258	0.258
	OB	0.938	0.941	0.945	0.937	1.712
	PO	1.277	1.272	1.287	1.259	2.008
	BT	2.467	2.950	2.461	2.455	3.980
	Avg. +/-	—	-2.73%	-78.02%	-6.28%	-55.70%
Deletion	AU	0.089	0.090	0.190	0.292	0.142
	SU	0.703	0.692	0.736	0.762	0.752
	YG	0.539	0.663	0.500	0.616	0.551
	OB	1.021	1.319	1.025	1.517	1.769
	PO	0.784	0.979	0.893	0.781	1.448
	BT	1.873	1.884	2.109	2.111	3.307
	Avg. +/-	—	-12.87%	-22.97%	-51.95%	-50.54%

overhead than deletions. This is because, for insertion, the sub-space vectors dominated by the original MCVs may become affected vectors, whereas for deletion, only the original MCVs themselves can become affected. As a result, insertion operations generate far more affected vectors to process. The runtime breakdown of MCM further shows that IS usually accounts for the major part of the running time under the single-edge update. This is because IS needs to traverse the graph and MCVs to identify the affected scope. In contrast, LC operates only within the compact affected scope. Since a single-edge update usually induces only limited changes in MCVs, LC typically converges within a few iterations.

Exp-2: Effect of optimizations. In this experiment, we evaluate the effectiveness of the optimizations proposed for LC in Section 6.3. We use FULL to denote LC equipped with all optimizations, and -OPT1 to -OPT4 to denote variants where each optimization is disabled individually. Table 3 reports the ablation results. Overall, disabling any individual optimization leads to degraded performance for both edge insertions and deletions. For insertions, removing OPT1–OPT4 results in average performance degradations of 2.73%, 78.02%, 6.28%, and 55.70%, respectively. For deletions, the corresponding degradations are 12.87%, 22.97%, 51.95%, and 50.54%, respectively. These results validate the effectiveness of our proposed optimization techniques.

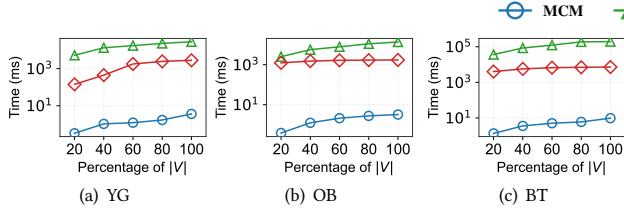


Figure 3: Effect of graph size (insertion)

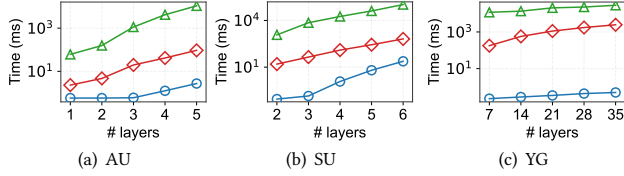


Figure 5: Effect of the number of layers (insertion)

Exp-3: Effect of graph size. To evaluate the impact of graph size, we construct induced subgraphs of different sizes by randomly sampling varying percentages of vertices, ranging from 20% to 100%. The results are shown in Figure 3 and Figure 4. Overall, the running time of all algorithms increases steadily as the graph size grows. For DEC and PARDEC, both methods perform multilayer core decomposition on the entire updated graph, making their computational overhead highly sensitive to the graph scale. For MCM, the running time increases moderately because larger graphs tend to induce a broader candidate affected scope. Nevertheless, MCM consistently outperforms DEC and PARDEC across all graph sizes, demonstrating its scalability for dynamic multilayer core maintenance.

Exp-4: Effect of the number of layers. To evaluate the effect of the number of layers, we vary the number of selected layers and extract the induced subgraph for each layer subset. Figure 5 and Figure 6 present the results on the three datasets with the largest numbers of layers. Overall, MCM consistently outperforms DEC and PARDEC across all layer configurations. The running time of DEC and PARDEC increases with more layers, since multilayer core decomposition becomes more computationally expensive on the multilayer graph with richer layer structures. For MCM, the running time typically increases steadily as the number of layers increases. This is because more layers lead to higher costs for dominance relation checking and larger search space during the refinement process. Nevertheless, MCM effectively alleviates such overhead through the binary state skipping strategy, which avoids exploring a large number of intermediate vectors during refinement. As a result, MCM remains efficient for multilayer core maintenance under different numbers of layers.

Exp-5: Memory analysis. In this experiment, we analyze the memory usage under edge insertion and deletion separately. Figure 7 shows the results, where GRAPH denotes the memory usage for graph loading. The memory usage remains nearly identical for edge insertions and deletions. This is because the main difference between the two operations lies in the identification of the affected scope, whose memory cost is negligible compared with that of loading the graph and original MCVs. Overall, MCM incurs only moderate memory overhead compared with GRAPH, demonstrating its memory efficiency for multilayer core maintenance. Moreover,

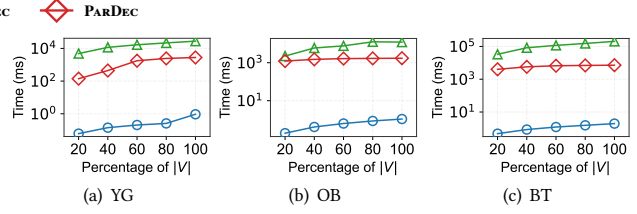


Figure 4: Effect of graph size (deletion)

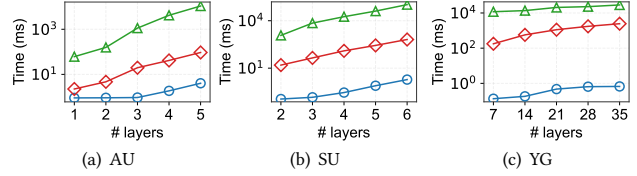


Figure 6: Effect of the number of layers (deletion)

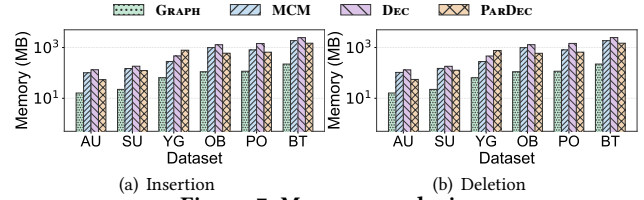


Figure 7: Memory analysis

MCM consistently consumes less memory than DEC, since it performs maintenance only within the affected scope rather than the whole graph. Compared with PARDEC, MCM exhibits slightly higher memory usage on some datasets because it needs to preload the original MCVs of all vertices for local updates during the maintenance process. However, MCM achieves better efficiency than PARDEC while incurring only slight additional memory overhead. These results demonstrate that MCM achieves an effective balance between memory efficiency and maintenance performance.

Exp-6: Effect of batch updates. We vary the number of updated edges $|\Delta E|$ from 100 to 500 and report the total running time in Figure 8 and Figure 9. The running times of DEC and PARDEC remain nearly unchanged as $|\Delta E|$ increases. This is because both methods perform decomposition on the entire graph and $|\Delta E| \ll |E|$. Hence, their costs are dominated by the graph size rather than $|\Delta E|$. In contrast, the running time of MCM-S increases steadily with $|\Delta E|$ as it processes the updated edges sequentially. MCM maintains relatively stable performance across different values of $|\Delta E|$ and outperforms MCM-S. The reason is that MCM employs a hybrid edge grouping strategy to support group-wise maintenance, allowing edges within the same group to be processed in parallel. These results demonstrate the effectiveness of the hybrid edge grouping strategy for batch updates.

Exp-7: Effect of the number of threads. We evaluate the performance of MCM with respect to the number of threads under different batch-edge update sizes. Figure 10 and Figure 11 report the results, where MCM-100 to MCM-500 denote MCM with $|\Delta E|$ ranging from 100 to 500. The results show that the running time of MCM decreases as the number of threads increases, demonstrating the effectiveness of MCM to process batch-edge updates in parallel. Moreover, MCM typically achieves faster speedup with larger

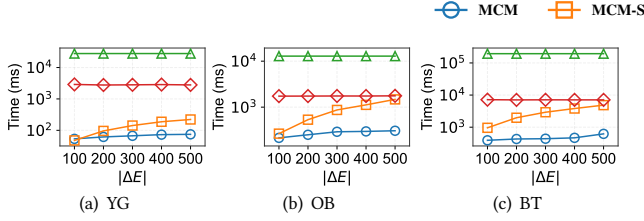


Figure 8: Effect of batch updates (insertion)

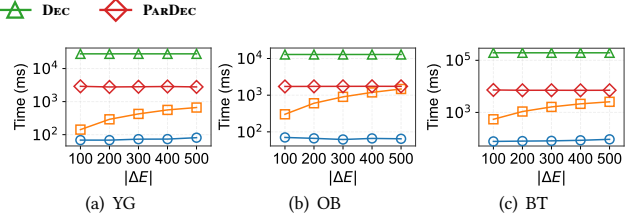


Figure 9: Effect of batch updates (deletion)

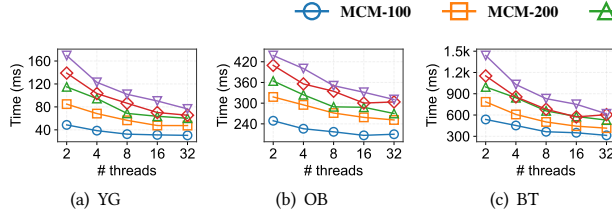


Figure 10: Effect of the number of threads (insertion)

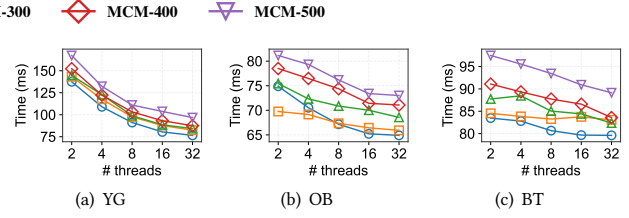


Figure 11: Effect of the number of threads (deletion)

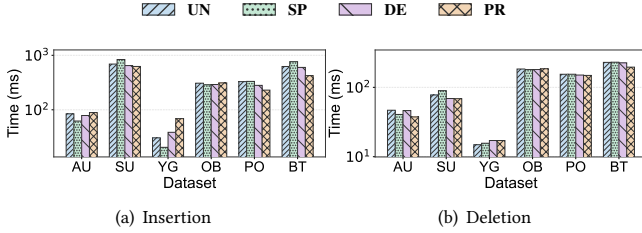


Figure 12: Effect of the distribution of updated edges

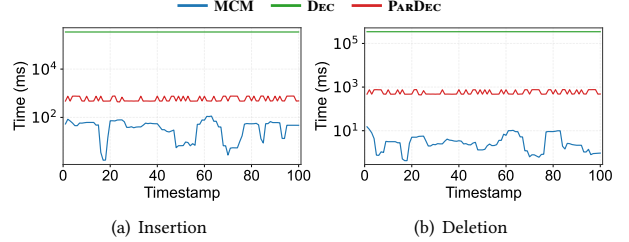


Figure 13: Case study on the temporal network

batch sizes for edge insertion, since more updated edges can be processed simultaneously. For edge deletion, the running time may occasionally fluctuate or change only slightly because most deleted edges can be processed efficiently, in which case the overhead of using more threads may outweigh the parallelization benefit. Overall, these results demonstrate that MCM can effectively leverage threads to accelerate the multilayer core maintenance.

Exp-8: Effect of the distribution of updated edges. We study the sensitivity of MCM to the distribution of updated edges. We generate 500 edges for each update, and the number of updated edges on layer ℓ is set to $500 \cdot \theta_\ell$. We consider four layer sampling strategies. Let $\text{rank}(\ell)$ denote the rank of layer ℓ in descending order of $|E_\ell|$, where a smaller rank indicates a denser layer. Specifically, UN samples edges uniformly across layers with $\theta_\ell = \frac{1}{|L|}$, SP favors sparser layers with $\theta_\ell = \frac{\text{rank}(\ell)}{\sum_{\ell' \in L} \text{rank}(\ell')}$, DE favors denser layers with $\theta_\ell = \frac{|L| - \text{rank}(\ell) + 1}{\sum_{\ell' \in L} (|L| - \text{rank}(\ell') + 1)}$, and PR samples edges proportionally with $\theta_\ell = \frac{|E_\ell|}{\sum_{\ell' \in L} |E_{\ell'}|}$. The results are shown in Figure 12. For most datasets, MCM exhibits similar running times across different sampling strategies, demonstrating its stability under different distributions of updated edges, except for edge insertion on the YG dataset. This is because YG has a large number of layers, and different sampling strategies may concentrate updated edges on different layers. Due to the structural heterogeneity among layers, these updates can lead to different affected scopes and maintenance costs. Overall, the results demonstrate the robustness of MCM to different distributions of updated edges.

Exp-9: Case study. We conduct a case study on the temporal network *MathOverflow* [43], where each vertex represents a user and each edge captures a temporal interaction such as answering a question or posting a comment. Following [7, 32], we sort all temporal edges by their earliest timestamps and divide the entire time span into five consecutive intervals, each of which is regarded as one layer in the multilayer graph. To simulate real network dynamics, we select the last 20 timestamps from each layer, obtaining 100 timestamps in total. The initial multilayer graph is constructed by excluding the edges associated with these selected timestamps. We then incrementally insert and delete the edges corresponding to each selected timestamp, and report the average update time per timestamp. As shown in Figure 13, MCM consistently outperforms DEC and PARDEC across all timestamps, demonstrating that MCM can efficiently support continuous updates in the real-world temporal network.

9 CONCLUSIONS

In this paper, we study the multilayer core maintenance problem on dynamic multilayer graphs. To solve this problem, we propose efficient maintenance algorithms that identify the affected scope for edge updates and compute MCVs via a local convergence procedure with a series of optimization techniques. In addition, we extend our algorithms to batch updates with a hybrid edge grouping strategy. Extensive experiments demonstrate the efficiency of our proposed algorithms. In the future, we would like to explore distributed multilayer core maintenance on large dynamic multilayer graphs.

REFERENCES

- [1] Askubuntu Network. 2026. <https://snap.stanford.edu/data/sx-askubuntu.html>.
- [2] N Azimi-Tafreshi, J Gómez-Gardenes, and SN Dorogovtsev. 2014. k-Core Percolation on Multiplex Networks. *Phys. Rev. E* 90, 3 (2014), 032816.
- [3] Wen Bai, Yadi Chen, and Di Wu. 2020. Efficient temporal core maintenance of massive graphs. *Inf. Sci.* 513 (2020), 324–340.
- [4] Wen Bai, Yadi Chen, Di Wu, Zhichuan Huang, Yipeng Zhou, and Chuan Xu. 2022. Generalized core maintenance of dynamic bipartite graphs. *Data Min. Knowl. Discov.* 36, 1 (2022), 209–239.
- [5] Nicola Barbieri, Francesco Bonchi, Edoardo Galimberti, and Francesco Gullo. 2015. Efficient and Effective Community Search. *Data Min. Knowl. Discov.* 29, 5 (2015), 1406–1433.
- [6] Ali Behrouz, Farnoosh Hashemi, and Laks V. S. Lakshmanan. 2022. FirmTruss Community Search in Multilayer Networks. *Proc. VLDB Endow.* 16, 3 (2022), 505–518.
- [7] Yu Chen, Qing Liu, Yifan Zhu, and Yunjun Gao. 2025. Efficient η -Threshold Maintenance in Dynamic Uncertain Graphs. In *ICDE*. 515–528.
- [8] Wanyun Cui, Yanghua Xiao, Haixun Wang, and Wei Wang. 2014. Local search of communities in large graphs. In *SIGMOD Conference*. 991–1002.
- [9] Jie Dai, Yixiao Liu, Yuan Li, Fazal Wahab, and Yuhai Zhao. 2025. I/O efficient core maintenance in large graphs. *Expert Syst. Appl.* 288 (2025), 128234.
- [10] Tim Dettmers, Pasquale Minervini, Pontus Stenatorp, and Sebastian Riedel. 2018. Convolutional 2D Knowledge Graph Embeddings. In *AAAI*. 1811–1818.
- [11] Xiangfei Fang, Chengying Huan, Heng Zhang, Yongchao Liu, Shaonan Ma, Yanjun Wu, and Chen Zhao. 2025. OTM: Efficient k-Order-Based Core Maintenance in Large-Scale Dynamic Hypergraphs. *ACM Trans. Knowl. Discov. Data* 19, 3 (2025), 79:1–79:33.
- [12] Yixiang Fang, Xin Huang, Lu Qin, Ying Zhang, Wenjie Zhang, Reynold Cheng, and Xuemin Lin. 2020. A survey of community search over big graphs. *VLDB J.* 29, 1 (2020), 353–392.
- [13] Yixiang Fang, Zhongran Wang, Reynold Cheng, Hongzhi Wang, and Jiafeng Hu. 2019. Effective and Efficient Community Search Over Large Directed Graphs. *IEEE Trans. Knowl. Data Eng.* 31, 11 (2019), 2093–2107.
- [14] FF-TW Network. 2026. <https://uinfolab.github.io/data/>.
- [15] Edoardo Galimberti, Francesco Bonchi, Francesco Gullo, and Tommaso Lanciano. 2020. Core Decomposition in Multilayer Networks: Theory, Algorithms, and Applications. *ACM Trans. Knowl. Discov. Data* 14, 1 (2020), 11:1–11:40.
- [16] Hongru Gao, Xiaofei Liao, Zhiyuan Shao, Kexin Li, Jiajie Chen, and Hai Jin. 2024. A survey on dynamic graph processing on GPUs: concepts, terminologies and systems. *Frontiers Comput. Sci.* 18, 4 (2024).
- [17] Sen Gao, Hongchao Qin, Rong-Hua Li, and Bingsheng He. 2023. Parallel Colorful h-star Core Maintenance in Dynamic Graphs. *Proc. VLDB Endow.* 16, 10 (2023), 2538–2550.
- [18] Bin Guo and Emil Sekerinski. 2024. Simplified algorithms for order-based core maintenance. *J. Supercomput.* 80, 13 (2024), 19592–19623.
- [19] Farnoosh Hashemi and Ali Behrouz. 2024. A Unified Core Structure in Multiplex Networks: From Finding the Densest Subgraph to Modeling User Engagement. In *KDD*. 1028–1039.
- [20] Farnoosh Hashemi, Ali Behrouz, and Laks V. S. Lakshmanan. 2022. FirmCore Decomposition of Multilayer Networks. In *WWW*. 1589–1600.
- [21] Qiang-Sheng Hua, Yuliang Shi, Dongxiao Yu, Hai Jin, Jiguo Yu, Zhipeng Cai, Xiuzhen Cheng, and Hanhua Chen. 2020. Faster Parallel Core Maintenance Algorithms in Dynamic Graphs. *IEEE Trans. Parallel Distributed Syst.* 31, 6 (2020), 1287–1300.
- [22] Qiang-Sheng Hua, Hongen Wang, Hai Jin, and Xuanhua Shi. 2024. Core Maintenance on Dynamic Graphs: A Distributed Approach Built on H-Index. *IEEE Trans. Big Data* 10, 5 (2024), 595–608.
- [23] Qiang-Sheng Hua, Xiaohui Zhang, Hai Jin, and Hong Huang. 2023. Revisiting Core Maintenance for Dynamic Hypergraphs. *IEEE Trans. Parallel Distributed Syst.* 34, 3 (2023), 981–994.
- [24] Hongxuan Huang, Qingyuan Linghu, Fan Zhang, Dian Ouyang, and Shiyu Yang. 2021. Truss Decomposition on Multilayer Graphs. In *IEEE BigData*. 5912–5915.
- [25] Xin Huang, Laks V. S. Lakshmanan, and Jianliang Xu. 2017. Community Search over Big Graphs: Models, Algorithms, and Opportunities. In *ICDE*. 1451–1454.
- [26] Daxin Jiang and Jian Pei. 2009. Mining frequent cross-graph quasi-cliques. *ACM Trans. Knowl. Discov. Data* 2, 4 (2009), 16:1–16:42.
- [27] Hai Jin, Na Wang, Dongxiao Yu, Qiang-Sheng Hua, Xuanhua Shi, and Xia Xie. 2018. Core Maintenance in Dynamic Graphs: A Parallel Approach Based on Matching. *IEEE Trans. Parallel Distributed Syst.* 29, 11 (2018), 2416–2428.
- [28] Tommaso Lanciano, Atsushi Miyauchi, Adriano Fazzone, and Francesco Bonchi. 2024. A Survey on the Densest Subgraph Problem and its Variants. *ACM Comput. Surv.* 56, 8 (2024), 208:1–208:40.
- [29] Jure Leskovec, Lada A Adamic, and Bernardo A Huberman. 2007. The Dynamics of Viral Marketing. *TWEB* 1, 1 (2007), 5–es.
- [30] Rong-Hua Li, Jeffrey Xu Yu, and Rui Mao. 2014. Efficient Core Maintenance in Large Dynamic Graphs. *IEEE Trans. Knowl. Data Eng.* 26, 10 (2014), 2453–2465.
- [31] Yuqi Li, Guosheng Zang, Chunyao Song, Xiaojie Yuan, and Tingjian Ge. 2024. Leveraging Semantic Information for Enhanced Community Search in Heterogeneous Graphs. *Data Sci. Eng.* 9, 2 (2024), 220–237.
- [32] Xuankun Liao, Qing Liu, Jiaxin Jiang, Byron Choi, Bingsheng He, and Jianliang Xu. 2025. Accelerating D-Core Maintenance over Dynamic Directed Graphs. In *ICDE*. 696–709.
- [33] Boge Liu, Fan Zhang, Chen Zhang, Wenjie Zhang, and Xuemin Lin. 2019. Core-Cube: Core Decomposition in Multilayer Graphs. In *WISE (Lecture Notes in Computer Science)*. 694–710.
- [34] Chuanyu Liu, Qing Liu, Chengyang Luo, and Yunjun Gao. 2026. PRISM: An LLM-Empowered Interactive System for Community Search on Multilayer Graphs. In *SIGMOD Companion*. 86–89.
- [35] Dandan Liu, Run-An Wang, Zhaonian Zou, and Xin Huang. 2024. Fast Multilayer Core Decomposition and Indexing. In *ICDE*. 2695–2708.
- [36] Dandan Liu and Zhaonian Zou. 2023. gCore: Exploring Cross-layer Cohesiveness in Multi-layer Graphs. *Proc. VLDB Endow.* 16, 11 (2023), 3201–3213.
- [37] Qing Liu, Xuliang Zhu, Xin Huang, and Jianliang Xu. 2021. Local Algorithms for Distance-generalized Core Decomposition over Large Dynamic Graphs. *Proc. VLDB Endow.* 14, 9 (2021), 1531–1543.
- [38] Xudong Liu and Zhaonian Zou. 2023. Common-Truss-Based Community Search on Multilayer Graphs. In *ADMA (3) (Lecture Notes in Computer Science)*. 277–291.
- [39] Yu Liu, Qi Luo, Yanwei Zheng, Zhipeng Cai, and Dongxiao Yu. 2025. Efficient traversal for core maintenance in large-scale dynamic hypergraphs. *WWW* 28, 4 (2025), 45.
- [40] Chengyang Luo, Qing Liu, Yunjun Gao, and Jianliang Xu. 2025. Synergetic Community Search over Large Multilayer Graphs. *Proc. VLDB Endow.* 18, 5 (2025), 1412–1424.
- [41] Wensheng Luo, Qiaoyuan Yang, Yixiang Fang, and Xu Zhou. 2023. Efficient Core Maintenance in Large Bipartite Graphs. *Proc. ACM Manag. Data* 1, 3 (2023), 208:1–208:26.
- [42] Manlio De Domenico. 2026. <https://manliodedomenico.com/data.php>.
- [43] Mathoverflow Network. 2026. <https://snap.stanford.edu/data/sx-mathoverflow.html>.
- [44] Ahmet Erdem Sariyüce, Bugra Gedik, Gabriela Jacques-Silva, Kun-Lung Wu, and Umit V. Çatalyürek. 2013. Streaming Algorithms for k-core Decomposition. *Proc. VLDB Endow.* 6, 6 (2013), 433–444.
- [45] Ahmet Erdem Sariyüce, Bugra Gedik, Gabriela Jacques-Silva, Kun-Lung Wu, and Umit V. Çatalyürek. 2016. Incremental k-core decomposition: algorithms and evaluation. *VLDB J.* 25, 3 (2016), 425–447.
- [46] Mauro Sozio and Aristides Gionis. 2010. The Community-search Problem and How to Plan a Successful Cocktail Party. In *KDD*. 939–948.
- [47] Longxu Sun, Xin Huang, Zheng Wu, and Jianliang Xu. 2024. Efficient Cross-layer Community Search in Large Multilayer Graphs. In *ICDE*. 2959–2971.
- [48] Zitan Sun, Xin Huang, Qing Liu, and Jianliang Xu. 2023. Efficient Star-based Truss Maintenance on Dynamic Graphs. *Proc. ACM Manag. Data* 1, 2 (2023), 133:1–133:26.
- [49] Superuser Network. 2026. <https://snap.stanford.edu/data/sx-superuser.html>.
- [50] Yifu Tang, Chengfei Liu, Lu Chen, Rui Zhou, and Jianxin Li. 2025. Finding Time-Proximity Communities in Temporal Heterogeneous Information Networks. *Proc. VLDB Endow.* 18, 13 (2025), 5740–5752.
- [51] Han Wang, Renjie Sun, Yongye Li, Chen Chen, Xiaoyang Wang, and Ying Zhang. 2025. Efficient Maximal Frequent Clique Enumeration in Multilayer Networks. In *DASFAA (3) (Lecture Notes in Computer Science)*. 169–185.
- [52] Na Wang, Dongxiao Yu, Hai Jin, Chen Qian, Xia Xie, and Qiang-Sheng Hua. 2017. Parallel Algorithm for Core Maintenance in Dynamic Graphs. In *ICDCS*. 2366–2371.
- [53] Run-An Wang, Dandan Liu, and Zhaonian Zou. 2024. FocusCore Decomposition of Multilayer Graphs. In *ICDE*. 2792–2804.
- [54] Tongfeng Weng, Mo Sha, Xu Zhou, Jingjing Lu, Kenli Li, and Kian-Lee Tan. 2025. Efficient Temporal Edge-Core Maintenance in Streaming Graphs. *Proc. VLDB Endow.* 19, 4 (2025), 726–739.
- [55] Qiaoyuan Yang, Wensheng Luo, Yixiang Fang, and Yuanyuan Zeng. 2026. Order-based Algorithms for Efficient Core Maintenance in Large Bipartite Graphs. *Proc. ACM Manag. Data* 4, 1 (2026), 1–26.
- [56] Rongjin Yang, Zijun Chen, and Wenyan Liu. 2026. Efficient Order-based Algorithms for Core Maintenance in Weighted Graph. *Inf. Sci.* 728 (2026), 122775.
- [57] Yikai Zhang, Jeffrey Xu Yu, Ying Zhang, and Lu Qin. 2017. A Fast Order-Based Approach for Core Maintenance. In *ICDE*. 337–348.
- [58] Wei Zhou, Hong Huang, Qiang-Sheng Hua, Dongxiao Yu, Hai Jin, and Xiaoming Fu. 2021. Core Decomposition and Maintenance in Weighted Graph. *WWW* 24, 2 (2021), 541–561.
- [59] Yuhang Zhou, Chuanyu Liu, Qing Liu, and Chengyang Luo. 2026. Core Maintenance on Dynamic Multilayer Graphs (technical report). <https://github.com/ZJU-DAILY/MCM/blob/main/MCM-TechnicalReport.pdf>.
- [60] Rong Zhu, Zhaonian Zou, and Jianzhong Li. 2018. Diversified Coherent Core Search on Multi-Layer Graphs. In *ICDE*. 701–712.

A BOUNDEDNESS ANALYSIS OF MULTILAYER CORE MAINTENANCE.

Let CHANGED be the set of vertices whose MCV set changes, and let $\|\text{CHANGED}\|_c$ denote the size of the c -hop neighbors of CHANGED for a fixed positive integer c . The multilayer core maintenance problem is bounded if there exists an algorithm whose cost is bounded by a polynomial function of $\|\text{CHANGED}\|_c$; otherwise, it is unbounded. We prove that the multilayer core maintenance is unbounded by reduction from D-core maintenance, which is known to be unbounded [32].

Specifically, given a directed graph $G_D = (V_D, E_D)$, a D-core, i.e., a (k, l) -core, is the maximal subgraph in which every vertex has in-degree at least k and out-degree at least l . We construct a two-layer directed multilayer graph $\mathcal{G} = (V_D, E, L)$ with $L = \{\ell_{in}, \ell_{out}\}$. For each directed edge $(u, w) \in E_D$, we create a copy of the same directed edge in each layer, i.e., we add (u, w, ℓ_{in}) and (u, w, ℓ_{out}) to E . The layer ℓ_{in} is used to evaluate incoming-neighbor constraints, where the degree of a vertex is defined as its in-degree, while the layer ℓ_{out} is used to evaluate outgoing-neighbor constraints, where the degree of a vertex is defined as its out-degree. Thus, for any $C \subseteq V_D$, the degrees of a vertex v on layers ℓ_{in} and ℓ_{out} in $\mathcal{G}[C]$ are exactly the in-degree and out-degree of v in the induced subgraph $G_D[C]$, respectively. Therefore, the (k, l) -core of G_D corresponds exactly to the multilayer k -core of $\mathcal{G}$ with $\mathbf{k} = (k, l)$ under the layer order ℓ_{in} and ℓ_{out} . The vertex v belongs to a (k, l) -core iff there exists $\mathbf{k}^* \in \Phi_{\mathcal{G}}(v)$ such that $\mathbf{k}^* \succeq (k, l)$. For a directed edge update in G_D , we can map it to two-layer edge updates in $\mathcal{G}$. Hence, if multilayer core maintenance were bounded, then D-core maintenance would be bounded, contradicting the known unboundedness of D-core maintenance [32]. Therefore, multilayer core maintenance is unbounded.

B THE PROOF OF LEMMA 1

LEMMA 1 (EDGE INSERTION BOUND). *Let $\mathcal{G}' = \mathcal{G} \cup \{e\}$, where e is inserted on layer ℓ . For any vertex $v \in V$ and any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$, there exists an original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ such that $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$, where $\mathbf{e}_\ell$ is the unit vector whose ℓ -th dimension is one and all other dimensions are zero.*

PROOF. We prove it by contradiction. Assume there exist a vertex $v \in V$ and an updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$ such that no original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ satisfies $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$. Since $\mathbf{k}'$ is an MCV of v in $\mathcal{G}'$, there exists a vertex set C with $v \in C$ such that $\mathcal{G}'[C]$ is a multilayer $\mathbf{k}'$ -core. After removing the inserted edge e , the degree of any vertex in C decreases by at most one on layer ℓ and remains unchanged on every layer $\ell' \neq \ell$. Therefore, every vertex in C still has degree at least $k'_\ell - 1$ on layer ℓ and at least $k'_{\ell'}$ on every other layer $\ell' \neq \ell$ in $\mathcal{G}[C]$. This implies that $\mathcal{G}[C]$ is also a multilayer $(\mathbf{k}' - \mathbf{e}_\ell)$ -core. Thus, $\mathbf{k}' - \mathbf{e}_\ell$ is a coreness vector of v in the original graph $\mathcal{G}$. By the definition of MCVs, there exists $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ such that $\mathbf{k} \succeq \mathbf{k}' - \mathbf{e}_\ell$. Therefore, $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$, contradicting the assumption. $\square$

C THE PROOF OF LEMMA 2

LEMMA 2 (SAME-LEVEL INSERTION TRANSITION). *If there is an insertion transition from $(x, \mathbf{k})$ to $(y, \mathbf{r})$ for an insertion on layer ℓ , then $k_\ell = r_\ell$.*

PROOF. By Definition 4, $\mathbf{r} = \mathbf{k} \wedge \mathbf{k}'$. Therefore, $\mathbf{k} \succeq \mathbf{r}$. Since no original MCV of x dominates $\mathbf{r} + \mathbf{e}_\ell$, we have $\mathbf{k} \not\succeq \mathbf{r} + \mathbf{e}_\ell$. Hence, we can infer that $k_\ell = r_\ell$. $\square$

D THE PROOF OF LEMMA 3

LEMMA 3 (EDGE INSERTION AFFECTED SCOPE). *Let $\mathcal{G}' = \mathcal{G} \cup \{e\}$. For any vertex $v \in V$ and any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$, there exists a vector $\bar{\mathbf{k}} \in \Phi_{\mathcal{G}}(v) \cup \{\mathbf{r} + \mathbf{e}_\ell \mid \mathbf{r} \in \mathcal{A}_\Phi(e, v)\}$ such that $\bar{\mathbf{k}} \succeq \mathbf{k}'$.*

PROOF. Let $e = (u, w, \ell)$ and consider any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$. If $\mathbf{k}'$ is dominated by an original MCV in $\Phi_{\mathcal{G}}(v)$, then the lemma holds directly. We now consider the remaining case where $\mathbf{k}'$ is not dominated by any original MCV of v . By Lemma 1, there exists an original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ such that $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$. Since $\mathbf{k}$ itself cannot dominate $\mathbf{k}'$, the dominance of $\mathbf{k}'$ must rely on the promotion on layer ℓ . Thus, the newly obtained support for $\mathbf{k}'$ must be introduced either by the inserted edge directly or by a propagated affected vector from a neighbor.

If the new support is introduced directly by the inserted edge, then v is an endpoint of e . Suppose that $v = u$ and w is the other endpoint of e . Let C be an updated multilayer $\mathbf{k}'$ -core containing v . If the inserted edge directly supports $\mathbf{k}'$ of v in $\mathcal{G}'[C]$, then the other endpoint w is also in $\mathcal{G}'[C]$. Thus, $\mathbf{k}'$ is a coreness vector of w in $\mathcal{G}'$, and is dominated by some updated MCV $\mathbf{h}' \in \Phi_{\mathcal{G}'}(w)$. By Lemma 1, there exists an original MCV $\mathbf{h} \in \Phi_{\mathcal{G}}(w)$ such that $\mathbf{h} + \mathbf{e}_\ell \succeq \mathbf{h}' \succeq \mathbf{k}'$. Let $\mathbf{r} = \mathbf{k} \wedge \mathbf{h}$, we next verify that $\mathbf{r} + \mathbf{e}_\ell$ dominates $\mathbf{k}'$. For any layer $i \neq \ell$, $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$ and $\mathbf{h} + \mathbf{e}_\ell \succeq \mathbf{k}'$ imply $k_i \geq k'_i$ and $h_i \geq k'_i$, so $r_i = \min\{k_i, h_i\} \geq k'_i$. For layer ℓ , they imply $k_\ell + 1 \geq k'_\ell$ and $h_\ell + 1 \geq k'_\ell$, so $r_\ell + 1 = \min\{k_\ell, h_\ell\} + 1 \geq k'_\ell$. Thus, $\mathbf{r} + \mathbf{e}_\ell \succeq \mathbf{k}'$. Since $\mathbf{k}'$ is not dominated by any original MCV of v , $\mathbf{r} + \mathbf{e}_\ell$ is also not dominated by any original MCV of v . Hence, the endpoint initialization inserts $\mathbf{r}$ into $\mathcal{A}_\Phi(e, v)$.

Otherwise, the new support is propagated from a neighbor $x \in N_{\mathcal{G}'}(v)$ whose affected subspace vector $\mathbf{h}$ has already been promoted. Let $\mathbf{r} = \mathbf{k} \wedge \mathbf{h}$. As in the direct case, since both $\mathbf{h} + \mathbf{e}_\ell$ and $\mathbf{k} + \mathbf{e}_\ell$ dominate $\mathbf{k}'$, $\mathbf{r} = \mathbf{h} \wedge \mathbf{k}$ gives $\mathbf{r} + \mathbf{e}_\ell \succeq \mathbf{k}'$. Moreover, no original MCV of x dominates $\mathbf{r} + \mathbf{e}_\ell$; otherwise, the support from x would already exist before the insertion and would not be a newly propagated support. No original MCV of v dominates $\mathbf{r} + \mathbf{e}_\ell$ either; otherwise, $\mathbf{k}'$ would be dominated by an original MCV of v . Therefore, the pair $(x, \mathbf{h})$ satisfies the insertion transition to $(v, \mathbf{r})$ and $\mathbf{r}$ is included into $\mathcal{A}_\Phi(e, v)$. Applying the same argument recursively to $(x, \mathbf{h})$ traces the newly available support backward along insertion transitions. The recursion terminates at an affected vector generated at an endpoint of the inserted edge, because the inserted edge is the only source of new support. Therefore, $\mathbf{r} \in \mathcal{A}_\Phi(e, v)$ and $\mathbf{r} + \mathbf{e}_\ell$ dominates $\mathbf{k}'$. $\square$

E THE PROOF OF LEMMA 4

LEMMA 4 (EDGE DELETION BOUND). *Let $\mathcal{G}' = \mathcal{G} \setminus \{e\}$, where e is deleted from layer ℓ . For any vertex $v \in V$ and any original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$, there exists an updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$ such that $\mathbf{k}' + \mathbf{e}_\ell \succeq \mathbf{k}$.*

PROOF. Since $\mathbf{k}$ is an MCV of v in $\mathcal{G}$, there exists a vertex set C with $v \in C$ such that $\mathcal{G}[C]$ is a multilayer $\mathbf{k}$ -core. After deleting e , the degree of any vertex in C decreases by at most one on layer

ℓ and remains unchanged on every other layer. Thus, $\mathcal{G}'[C]$ is a multilayer $\mathbf{k}^{(\ell,-1)}$ -core. Hence, $\mathbf{k}^{(\ell,-1)}$ is a coreness vector of v in $\mathcal{G}'$ and is dominated by some MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$, i.e., $\mathbf{k}' \succeq \mathbf{k}^{(\ell,-1)}$. Therefore, $\mathbf{k}' + \mathbf{e}_\ell \succeq \mathbf{k}$. $\square$

F THE PROOF OF LEMMA 5

LEMMA 5 (SAME-LEVEL DELETION TRANSITION). *If there is a deletion transition from $(x, \mathbf{k})$ to $(y, \mathbf{k}')$ for a deletion on layer ℓ , then $k_\ell = k'_\ell$.*

PROOF. By the definition of deletion transition, $\mathbf{k} \succeq \mathbf{k}'$, so $k_\ell \geq k'_\ell$. If $k_\ell > k'_\ell$, then $\mathbf{k}^{(\ell,-1)} \succeq \mathbf{k}'$. Since $\mathbf{k} \in \Phi_{\mathcal{G}}(x)$, it contradicts the condition that $\nexists \mathbf{h} \in \Phi_{\mathcal{G}}(x), \mathbf{h}^{(\ell,-1)} \succeq \mathbf{k}'$. Therefore, $k_\ell = k'_\ell$. $\square$

G THE PROOF OF LEMMA 6

LEMMA 6 (EDGE DELETION AFFECTED SCOPE). *Let $\mathcal{G}' = \mathcal{G} \setminus \{e\}$. For any vertex $v \in V$, if an original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ may become invalid after deleting e , then $\mathbf{k} \in \mathcal{A}_\Phi(e, v)$.*

PROOF. Assume that an MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ may become invalid after deleting e . If the invalidation of $\mathbf{k}$ is caused directly by the deleted edge, then v is an endpoint and the other endpoint of the deleted edge has an MCV dominating $\mathbf{k}$. Thus, $\mathbf{k}$ is contained in the affected vector set of v .

Otherwise, the invalidation of $\mathbf{k}$ must be caused by a neighbor x whose original MCV dominates $\mathbf{k}$ in the original graph but can no longer be guaranteed to dominate $\mathbf{k}$ after the deletion. Let $\mathbf{k}_x \in \Phi_{\mathcal{G}}(x)$ be an affected vector of x with $\mathbf{k}_x \succeq \mathbf{k}$. If there exists an MCV $\mathbf{h} \in \Phi_{\mathcal{G}}(x)$ such that $\mathbf{h}^{(\ell,-1)} \succeq \mathbf{k}$, then by Lemma 4, x would have an updated MCV dominating $\mathbf{h}^{(\ell,-1)}$, and hence still dominating $\mathbf{k}$. This contradicts the fact that the dominance relation from x may be lost. Therefore, no such $\mathbf{h}$ exists. Since $\mathbf{k}_x \succeq \mathbf{k}$ and no original MCV of x still dominates $\mathbf{k}$ after decreasing one on layer ℓ , there is a deletion transition from $(x, \mathbf{k}_x)$ to $(v, \mathbf{k})$. Applying the same argument recursively to $(x, \mathbf{k}_x)$ yields a sequence of deletion transitions, and the recursion terminates at a directly affected vector generated at an endpoint of the deleted edge. Thus, there exists a deletion affected path of e that reaches $(v, \mathbf{k})$. Therefore, $\mathbf{k} \in \mathcal{A}_\Phi(e, v)$. $\square$

H THE PROOF OF LEMMA 7

LEMMA 7 (VALIDATION RULE). *If every vector $\bar{\mathbf{k}} \in \bar{\Phi}_{\mathcal{G}}(v)$ is valid for any $v \in V$, then $\bar{\mathbf{k}}$ is a coreness vector of v in $\mathcal{G}$.*

PROOF. For any $\bar{\mathbf{k}} \in \bar{\Phi}_{\mathcal{G}}(v)$, let $S_{\bar{\mathbf{k}}} = \{u \in V \mid \exists \bar{\mathbf{k}}' \in \bar{\Phi}_{\mathcal{G}}(u), \bar{\mathbf{k}}' \succeq \bar{\mathbf{k}}\}$. Notably, $v \in S_{\bar{\mathbf{k}}}$. For any $u \in S_{\bar{\mathbf{k}}}$, there exists a valid vector $\bar{\mathbf{k}}' \in \bar{\Phi}_{\mathcal{G}}(u)$ with $\bar{\mathbf{k}}' \succeq \bar{\mathbf{k}}$. By validity, for each layer ℓ , u has at least $\bar{k}'_\ell \geq \bar{k}_\ell$ neighbors whose upper-bound sets contain vectors dominating $\bar{\mathbf{k}}'$, and hence also dominating $\bar{\mathbf{k}}$. Therefore, these neighbors belong to $S_{\bar{\mathbf{k}}}$ and the subgraph induced by $S_{\bar{\mathbf{k}}}$ is a multilayer $\bar{\mathbf{k}}$ -core containing v . $\square$

I THE PROOF OF LEMMA 8

LEMMA 8 (SAFE REFINEMENT). *Suppose that for every vertex $x \in V$, every MCV of x is dominated by at least one vector in $\bar{\Phi}_{\mathcal{G}}(x)$. Given an invalid vector $\bar{\mathbf{k}} \in \bar{\Phi}_{\mathcal{G}}(v)$ and a safe refinement set $\mathcal{R}(\bar{\mathbf{k}})$, let*

$\bar{\Phi}'_{\mathcal{G}}(v) = \bar{\Phi}_{\mathcal{G}}(v) \setminus \{\bar{\mathbf{k}}\} \cup \mathcal{R}(\bar{\mathbf{k}})$. *Then every MCV of v is dominated by at least one vector in $\bar{\Phi}'_{\mathcal{G}}(v)$.*

PROOF. Consider any MCV $\mathbf{k}^* \in \Phi_{\mathcal{G}}(v)$. By the definition of $\bar{\Phi}_{\mathcal{G}}(v)$, there exists a vector $\mathbf{h} \in \bar{\Phi}_{\mathcal{G}}(v)$ such that $\mathbf{h} \succeq \mathbf{k}^*$. If $\mathbf{h} \neq \bar{\mathbf{k}}$, then $\mathbf{h}$ remains in $\bar{\Phi}'_{\mathcal{G}}(v)$, so $\mathbf{k}^*$ is still dominated by at least one vector in $\bar{\Phi}'_{\mathcal{G}}(v)$; otherwise, $\mathbf{h} = \bar{\mathbf{k}}$, i.e., $\bar{\mathbf{k}} \succeq \mathbf{k}^*$. Since the invalid vector $\bar{\mathbf{k}}$ cannot be an MCV $\mathbf{k}^*$, $\bar{\mathbf{k}} \neq \mathbf{k}^*$. By the definition of safe refinement, there exists a vector $\bar{\mathbf{k}}' \in \mathcal{R}(\bar{\mathbf{k}})$ such that $\bar{\mathbf{k}}' \succeq \mathbf{k}^*$. Since $\bar{\mathbf{k}}' \in \bar{\Phi}'_{\mathcal{G}}(v)$, $\mathbf{k}^*$ remains dominated after the refinement. Therefore, $\mathbf{k}^*$ is still dominated by at least one vector in $\bar{\Phi}'_{\mathcal{G}}(v)$. $\square$

J THE PROOF OF LEMMA 9

LEMMA 9 (DOMINANCE INHERITANCE). *Consider an upper-bound vector of v refined from $\bar{\mathbf{k}}$ to $\bar{\mathbf{k}}^{new}$. For any neighbor u of v , every $\bar{\mathbf{k}}' \in \text{Dom}(\bar{\mathbf{k}}, v, u)$ satisfies $\bar{\mathbf{k}}' \succ \bar{\mathbf{k}}^{new}$.*

PROOF. By refinement rule, $\bar{\mathbf{k}} \succ \bar{\mathbf{k}}^{new}$. Since any vector $\bar{\mathbf{k}}' \in \text{Dom}(\bar{\mathbf{k}}, v, u)$ satisfies $\bar{\mathbf{k}}' \succeq \bar{\mathbf{k}}$, we have $\bar{\mathbf{k}}' \succ \bar{\mathbf{k}}^{new}$ by transitivity. Hence, every vector in $\text{Dom}(\bar{\mathbf{k}}, v, u)$ dominates $\bar{\mathbf{k}}^{new}$. $\square$

K THE PROOF OF LEMMA 10

LEMMA 10 (REPEATED VECTOR FILTERING). *The ordered splitting rule generates each vector at most once.*

PROOF. Consider any vector generated from an initial invalid vector. It can be uniquely described by a decreasing vector $\mathbf{d} = (d_1, \dots, d_{|L|})$, where d_i denotes how many times the value on layer i has been decreased. That is, once $\mathbf{d}$ is fixed, the resulting vector is also fixed. Under the ordered splitting rule, decreases must be applied in non-decreasing order of layer indices. Therefore, for a given decreasing vector $\mathbf{d}$, there is only one valid splitting path: first decrease layer 1 for d_1 times, then layer 2 for d_2 times, and so on until layer $|L|$. Any different order would apply a decrease to some layer j before a later decrease to a smaller layer $i < j$, which violates the ordered splitting rule. Hence, each decreasing vector can be generated by at most one valid splitting path. Since each generated vector is uniquely determined by its decreasing vector, the ordered splitting rule generates each vector at most once. $\square$

L THE PROOF OF LEMMA 11

LEMMA 11 (VALIDITY HIERARCHY). *Consider two upper-bound vectors $\bar{\mathbf{k}}$ and $\bar{\mathbf{k}}'$ of the same vertex, where $\bar{\mathbf{k}} \succeq \bar{\mathbf{k}}'$. If $\bar{\mathbf{k}}$ is valid, $\bar{\mathbf{k}}'$ is valid. If $\bar{\mathbf{k}}'$ is invalid, $\bar{\mathbf{k}}$ is invalid.*

PROOF. Assume that $\bar{\mathbf{k}}$ is valid. By the validation rule, for every layer $\ell \in L$, there exist at least $\bar{k}_\ell$ neighbors on layer ℓ whose upper-bound vectors dominate $\bar{\mathbf{k}}$. Therefore, these neighbors also support $\bar{\mathbf{k}}'$ and $\bar{\mathbf{k}}'$ is valid. Suppose that $\bar{\mathbf{k}}'$ is invalid. If $\bar{\mathbf{k}}$ were valid, $\bar{\mathbf{k}}'$ would also be valid as proved above, which contradicts the assumption that $\bar{\mathbf{k}}'$ is invalid. Therefore, $\bar{\mathbf{k}}$ must be invalid. $\square$

M THE PROOF OF LEMMA 12

LEMMA 12 (SUBTREE DOMINANCE). *Consider a vector $\bar{\mathbf{k}}$ in a splitting tree of the refinement phase and two child subtrees T_i and T_j*

generated by decreasing the value of $\bar{k}$ on layers i and j with $i < j$, respectively. Then no vector in T_i dominates any vector in T_j . In contrast, a vector in T_j may dominate a vector in T_i .

PROOF. Every vector in T_i is generated by a splitting sequence that first decreases layer i . Thus, for any $\mathbf{x} \in T_i$, we have $x_i < \bar{k}_i$. By the ordered splitting rule, every vector in T_j can only decrease value on the dimension with indices at least j . Since $i < j$, the value on the i -th dimension cannot be decreased in T_j . Hence, for any $\mathbf{y} \in T_j$, we have $y_i = \bar{k}_i$. Therefore, for any $\mathbf{x} \in T_i$ and $\mathbf{y} \in T_j$, we have $x_i < y_i$, which implies $\mathbf{x} \not\geq \mathbf{y}$. Next, we show that reverse dominance may occur. Let $a_i < \bar{k}_i$ and $a_j < \bar{k}_j$ be two decreased dimension values. Consider a vector $\mathbf{y} \in T_j$ with $y_j = a_j$ and $y_\ell = \bar{k}_\ell$ for every $\ell \neq j$. Let $\mathbf{x} \in T_i$ be a vector obtained by first decreasing the value on layer i to a_i and later decreasing the value on layer j to a_j , while keeping the remaining dimensions unchanged. Then $\mathbf{y}$ and $\mathbf{x}$ have the same values on all layers except layer i . Besides, $y_i = \bar{k}_i > a_i = x_i$. Therefore, $\mathbf{y} \geq \mathbf{x}$. $\square$

N THE PROOF OF LEMMA 13

LEMMA 13 (SUBSPACE COVERAGE). *If the splitting tree rooted at $\bar{\mathbf{r}}$ has been fully explored, then no vector in $\mathcal{D}(\bar{\mathbf{r}})$ can produce a new maximal valid vector.*

PROOF. Any descendant of a vector $\bar{\mathbf{k}} \in \mathcal{D}(\bar{\mathbf{r}})$ is also dominated by $\bar{\mathbf{r}}$, and is therefore also included in $\mathcal{D}(\bar{\mathbf{r}})$. Since the splitting tree rooted at $\bar{\mathbf{r}}$ has been fully explored, all maximal valid vectors in this subspace have already been found. Thus, every valid vector generated from $\bar{\mathbf{k}}$ is either one of these maximal valid vectors or is dominated by one of them, and cannot become a new maximal valid vector. $\square$

O THE PROOF OF LEMMA 14

LEMMA 14 (DISJOINT EDGE GROUP BOUND). *Let $B \subseteq \Delta E_\ell$ be a disjoint edge group updated on layer ℓ . If B is inserted, then for any vertex $v \in V$ and any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G} \cup B}(v)$, there exists an original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ such that $\mathbf{k} + \mathbf{e}_\ell \geq \mathbf{k}'$. If B is deleted, then for any vertex $v \in V$ and any original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$, there exists an updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G} \setminus B}(v)$ such that $\mathbf{k}' + \mathbf{e}_\ell \geq \mathbf{k}$.*

PROOF. For insertion, consider any $\mathbf{k}' \in \Phi_{\mathcal{G} \cup B}(v)$ and a vertex set C containing v such that $(\mathcal{G} \cup B)[C]$ is a multilayer $\mathbf{k}'$ -core. Since B is a disjoint edge group, each vertex in C is incident to at most one inserted edge in B . After removing B , the degree of every vertex decreases by at most one on layer ℓ and remains unchanged on all other layers. Thus, $\mathcal{G}[C]$ is a multilayer $\mathbf{k}'^{(\ell, -1)}$ -core. Therefore, $\mathbf{k}'^{(\ell, -1)}$ is a coreness vector of v in $\mathcal{G}$ and is dominated by some MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$. Hence, $\mathbf{k} + \mathbf{e}_\ell \geq \mathbf{k}'$.

For deletion, consider any $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ and a vertex set C containing v such that $\mathcal{G}[C]$ is a multilayer $\mathbf{k}$ -core. Since B is a disjoint edge group, after deleting B , the degree of every vertex in C decreases by at most one on layer ℓ and remains unchanged on all other layers. Therefore, $(\mathcal{G} \setminus B)[C]$ is a multilayer $\mathbf{k}^{(\ell, -1)}$ -core and $\mathbf{k}^{(\ell, -1)}$ is a coreness vector of v in $\mathcal{G} \setminus B$, which indicates that $\mathbf{k}^{(\ell, -1)}$ is dominated by some $\mathbf{k}' \in \Phi_{\mathcal{G} \setminus B}(v)$. Hence, $\mathbf{k}' + \mathbf{e}_\ell \geq \mathbf{k}$. $\square$

P THE PROOF OF LEMMA 15

LEMMA 15 (JOINT EDGE GROUP BOUND). *Let $B \subseteq \Delta E_\ell$ be a joint edge group. If B is inserted, then for any vertex $v \in V$ and any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G} \cup B}(v)$, there exists an original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ such that $\mathbf{k} + \mathbf{e}_\ell \geq \mathbf{k}'$. If B is deleted, then for any vertex $v \in V$ and any original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$, there exists an updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G} \setminus B}(v)$ such that $\mathbf{k}' + \mathbf{e}_\ell \geq \mathbf{k}$.*

PROOF. We first consider insertion. For any common endpoint $w \in C_B$ and any incident edge $(u, w, \ell) \in B$, let $\mathbf{h} \in \Phi_{\mathcal{G}}(w)$, $\mathbf{q} \in \Phi_{\mathcal{G}}(u)$, and $\mathbf{r} = \mathbf{h} \wedge \mathbf{q}$. By Definition 8, we have $h_\ell \geq \mu_\ell(w) > \eta_\ell(u) \geq q_\ell$. Hence, $r_\ell = q_\ell$ and $\mathbf{h} \geq \mathbf{r} + \mathbf{e}_\ell$. Since $\mathbf{r} + \mathbf{e}_\ell$ is dominated by an original MCV of w , w cannot become an affected vertex under the insertion transition. Therefore, the MCVs of w will not be affected by inserting edges in B . Now consider any $\mathbf{k}' \in \Phi_{\mathcal{G} \cup B}(v)$ and a vertex set C containing v such that $(\mathcal{G} \cup B)[C]$ is a $\mathbf{k}'$ -core. For each common endpoint $w \in C \cap C_B$, since the MCVs of w remain unchanged after inserting B , there exists an original MCV $\mathbf{h}_w \in \Phi_{\mathcal{G}}(w)$ such that $\mathbf{h}_w \geq \mathbf{k}'$. Let C_w be a vertex set containing w such that $\mathcal{G}[C_w]$ is a $\mathbf{h}_w$ -core. Then $\mathcal{G}[C_w]$ is also a $\mathbf{k}'$ -core. Let $\bar{C} = C \cup \bigcup_{w \in C \cap C_B} C_w$. After removing B , every vertex in $C \setminus C_B$ loses at most one incident edge on layer ℓ , because it is incident to at most one edge in B by the definition of C_B , and its degree on all other layers remains unchanged. Since $\mathcal{G}[C_w]$ is a $\mathbf{k}'$ -core, $\mathcal{G}[\bar{C}]$ is a $\mathbf{k}'^{(\ell, -1)}$ -core containing v . Thus, $\mathbf{k}'^{(\ell, -1)}$ is a coreness vector of v in $\mathcal{G}$. Therefore, it is dominated by some $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$. Hence, $\mathbf{k} + \mathbf{e}_\ell \geq \mathbf{k}'$.

For deletion, a common endpoint w can be directly affected by deleting (u, w, ℓ) only if some MCV of u dominates an MCV of w , which is impossible because every MCV of w has a larger value on layer ℓ than every MCV of u by Definition 8. Moreover, by Lemma 5, deletion transitions only appear between vectors with the same value on layer ℓ . Hence, the common endpoints remain unaffected. Now consider any original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ and a vertex set C containing v such that $\mathcal{G}[C]$ is a $\mathbf{k}$ -core. For each common endpoint $w \in C \cap C_B$, since the MCVs of w remain unchanged after deleting B , there exists an updated MCV $\mathbf{h}'_w \in \Phi_{\mathcal{G} \setminus B}(w)$ such that $\mathbf{h}'_w \geq \mathbf{k}$. Let C'_w be a vertex set containing w such that $(\mathcal{G} \setminus B)[C'_w]$ is a $\mathbf{h}'_w$ -core. Then $(\mathcal{G} \setminus B)[C'_w]$ is also a $\mathbf{k}$ -core. Let $\bar{C} = C \cup \bigcup_{w \in C \cap C_B} C'_w$. After deleting B , every vertex in $C \setminus C_B$ loses at most one incident edge on layer ℓ , because it is incident to at most one edge in B , and its degree on all other layers remains unchanged. Since $(\mathcal{G} \setminus B)[C'_w]$ is a $\mathbf{k}$ -core, $(\mathcal{G} \setminus B)[\bar{C}]$ is a $\mathbf{k}^{(\ell, -1)}$ -core containing v . Thus, $\mathbf{k}^{(\ell, -1)}$ is a coreness vector of v in $\mathcal{G} \setminus B$ and is dominated by some $\mathbf{k}' \in \Phi_{\mathcal{G} \setminus B}(v)$. Hence, $\mathbf{k}' + \mathbf{e}_\ell \geq \mathbf{k}$. $\square$

Q CORRECTNESS ANALYSIS OF ALGORITHM 3

We prove that Algorithm 3 terminates and returns the MCV set when validation follows Lemma 7 and each refinement is safe.

LEMMA 16 (TERMINATION). *Algorithm 3 terminates after a finite number of updates.*

PROOF. For each vertex v , let $\mathcal{D}_t(v) = \{\mathbf{k} \in \mathbb{N}^{|\mathcal{L}|} \mid \exists \bar{\mathbf{k}} \in \bar{\Phi}_{\mathcal{G}, t}(v), \bar{\mathbf{k}} \geq \mathbf{k}\}$ be the subspace vector set of its upper-bound

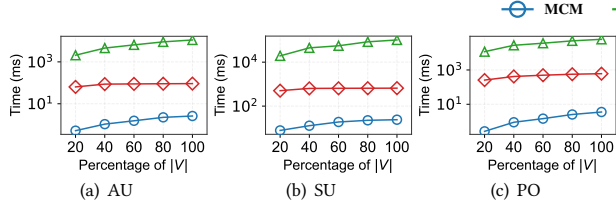


Figure 14: Effect of graph size (insertion)

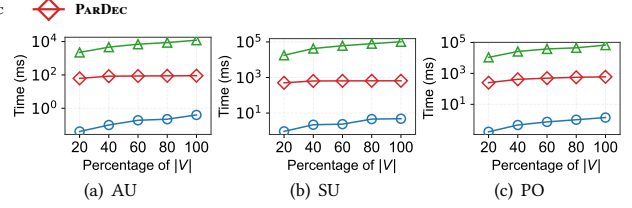


Figure 15: Effect of graph size (deletion)

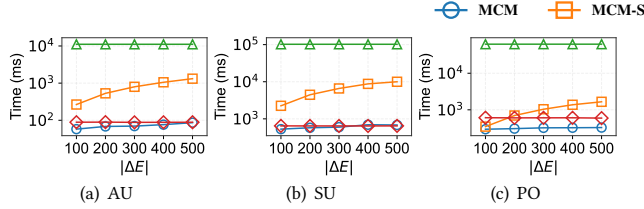


Figure 16: Effect of batch updates (insertion)

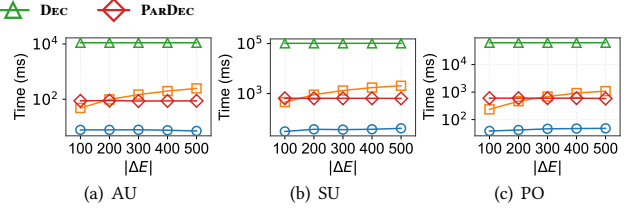


Figure 17: Effect of batch updates (deletion)

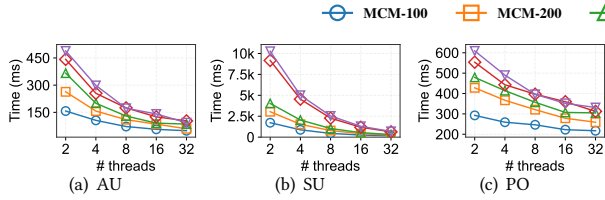


Figure 18: Effect of the number of threads (insertion)

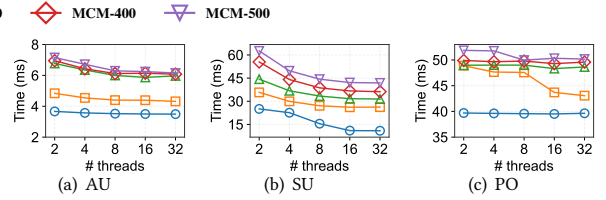


Figure 19: Effect of the number of threads (deletion)

set $\bar{\Phi}_{\mathcal{G},t}(v)$ at time t . Initially, $\mathcal{D}_0(v)$ is finite. For each update, an invalid vector $\bar{\mathbf{k}}$ is replaced by a safe refinement set and all new vectors in the safe refinement set are strongly dominated by $\bar{\mathbf{k}}$. Therefore, no vector outside the subspace vector set is added, and $\bar{\mathbf{k}}$ is removed from this set. Thus, $\sum_{v \in \mathcal{A}(e)} |\mathcal{D}_t(v)|$ decreases after every update and is bounded below by zero. Finally, Algorithm 3 terminates after a finite number of updates. $\square$

LEMMA 17 (MAXIMALITY PRESERVATION). *During Algorithm 3, for every vertex $v \in \mathcal{A}(e)$ and every MCV $\mathbf{k}^* \in \Phi_{\mathcal{G}}(v)$, there exists a vector $\bar{\mathbf{k}} \in \bar{\Phi}_{\mathcal{G}}(v)$ such that $\bar{\mathbf{k}} \succeq \mathbf{k}^*$.*

PROOF. The claim holds initially by the definition of the upper-bound vector set. By Lemma 8, each refinement preserves this claim. Dominance filtering also preserves it because a filtered vector is dominated by a retained vector. The lemma follows by induction over all updates. $\square$

THEOREM 1 (CORRECTNESS). *Upon termination of Algorithm 3, $\bar{\Phi}_{\mathcal{G}}(v) = \Phi_{\mathcal{G}}(v)$ holds for every vertex $v \in V$.*

PROOF. For every $v \notin \mathcal{A}(e)$, $\bar{\Phi}_{\mathcal{G}}(v) = \Phi_{\mathcal{G}}(v)$ holds by the affected scope identification procedure. For every $v \in \mathcal{A}(e)$, upon termination, every vector in every $\bar{\Phi}_{\mathcal{G}}(v)$ is valid. By Lemma 7, every retained vector is a valid coreness vector. For any $v \in \mathcal{A}(e)$ and $\mathbf{k}^* \in \Phi_{\mathcal{G}}(v)$, Lemma 17 gives a vector $\bar{\mathbf{k}} \in \bar{\Phi}_{\mathcal{G}}(v)$ with $\bar{\mathbf{k}} \succeq \mathbf{k}^*$. Since $\mathbf{k}^*$ should be maximal, we have $\bar{\mathbf{k}} = \mathbf{k}^*$. Finally, dominance filtering ensures that the vectors in $\bar{\Phi}_{\mathcal{G}}(v)$ remain mutually incomparable. Thus, $\bar{\Phi}_{\mathcal{G}}(v) = \Phi_{\mathcal{G}}(v)$ holds for every $v \in \mathcal{A}(e)$. $\square$

R ADDITIONAL EXPERIMENTAL RESULTS

In this section, we provide additional experimental results on the remaining datasets, i.e., AU, SU, and PO. Figures 14 and 15 report the effect of graph size. Figures 16 and 17 show the performance under batch updates. Figures 18 and 19 demonstrate the results under different numbers of threads.